Chapter 1

Languages, Encodings and Problems

1.1 Definitions

- An *alphabet* (Σ) is a non-empty, finite set. The elements of Σ are called *symbols / characters*.
- Let Σ be an alphabet. Σ^* is the set of all string over Σ consisting of finitely many symbol:

$$\Sigma^* = \{a_1 a_2 \dots a_n : n \in \mathbb{N}, a_i \in \Sigma, \forall i\}$$

- *Reversal of string*: $w = a_1 a_2 \dots a_n, w^R = a_n a_{n-1} \dots a_1$.
- *Star operation on a language*:

$$L^* = \bigcup_{n \in \mathbb{N}} L^n$$

  Star can be interchanged with reversal: $(L^*)^R = (L^R)^*$
  Star cannot be distributed over intersection: $(L_1 \cap L_2)^* \neq L_1^* \cap L_2^*$ (counterexample: $L_1 = \{a\}, L_2 = \{aa\}$)
- An *encoding* of the element of A using Σ is an injective function $Enc : A \to \Sigma^*$.
- Any function $f : \Sigma^* \to \Sigma^*$ is a *computational problem* over alphabet Σ.
  Computational problem as mapping instances to solutions: $g : I \to S$ (*error string*).
- Any function $f : \Sigma^* \to \{0, 1\}$ is a *decision problem* over alphabet Σ.
  <u>*Correspondence between decision problems and languages*</u>: Define $L \subseteq \Sigma^*$ to be the set of all words in Σ^* that f maps to 1, then L is the language corresponding to the decision problem. Similarly, take $l \subseteq \Sigma^*$, define corresponding decision problem $f : \Sigma^* \to \{0, 1\}$ as $f(w) = 1$ iff $w \in L$.

1.2 Exercises

1. Give a simple English description for the language $\{011, 101, 110\}^*$.
   - The set of finite length string consisting of 011's, 191's, 110's.
2. Let Σ be an alphabet. For which language $L \subseteq \Sigma^*$ is it true that L^* is infinite?
   - Any language except for $\varnothing$ and $\{\epsilon\}$.
3. Balanced Parentheses:
   Language L is recursively defined as
   - $() \in L$

- $x \in L \Rightarrow (x) \in L$ (WRAP)
- $x, y \in L \Rightarrow xy \in L$ (CONCAT)

Prove that a string $x \in L$ iff it satisfies language K:

- The string is non-empty
- The total number of open and closed parens are equal
- When scanning the string from left to right, at all points in time # open $\geq$ # closed

Proof.
- Part 1: $L \subseteq K$: We'll prove this by structural induction on L that for any $x \in L, x \in K$ as well.
  - **(BC)** () is a nonempty string, with 1 open and closed, scan (✓)
  - **(IS)** Consider some string $s \in L$ that was created either used WRAP or CONCAT rule. This means either $s = (x)$ for some $x \in L$ or $s = xy$ for some $x, y \in L$.
    - Case 1: $s = (x), x \in L$: Assume $x \in K$ **(IH)**. WTS $s \in K$.
      - non-empty ✓
      - By **IH**, $x \in K$, $\#open_x = \#closed_x \Rightarrow \#open_x + 1 = \#closed_x + 1$ ✓
      - AFSOC that at some point of scanning we had $\#open < \#closed$. This won't happen at reading the last closed paren because from the above we conclude $\#open = \#closed$ at the end. This should happen when scanning $(x$. Now scan x, at the same point we should have $\#open < \#closed$, because x has fewer open parens than $(x$. This contradicts **IH** that $x \in K$ ✓.
    - Case 2: $s = xy, x, y \in L$: Assume $x, y \in K$ **(IH)**. WTS $s \in K$
      - non-empty ✓
      - By **IH**, $\#open_x = \#closed_x, \#open_y = \#closed_y \Rightarrow \#open_x + \#open_y = \#closed_x + \#closed_y$ ✓
      - AFSOC $\#open < \#closed$ at some point during the scan. The closed paren that made this happen for the first time is either in x or in y. If it's in x then x doesn't satisfy property 3 of K. If it's in y then by the end of scanning x, $\#open$ scanned $= \#closed$ scanned, then $\#open_y$ scanned $< \#closed_y$ scanned, which is also a contradiction to $y \in K$ ✓
- Part 2: $K \subseteq L$: We'll prove this by strong induction on the length of the string. WTS for any $s \in K, s \in L$.
  - **(BC)** Smallest valid string length is 2 and is (). (✓)
  - **(IH)** Suppose for some $n \in \mathbb{N}(n \geq 3)$, all strings in K of length less than n also belong to L.
  - **(IS)** Consider some string $s \in K$ of length n. As we scan the string from left to right, we case on the value $v = \#open - \#closed$:
    - Case 1: $v \neq 0$ until the very last paren of s. Let a be the substring between the first and last char of s. $a \in K$ because it's non-empty (✓), $\#open_s = \#closed_s \Rightarrow \#open_s - 1 = \#closed_s - 1 = \#open_a = \#closed_a$ (✓), $v > 0 \Rightarrow v - 1 \geq 0$ (✓). Then by **IH**, $a \in K \Rightarrow a \in L$. By WRAP, $(a) \in L$.
    - Case 2: $v = 0$ somewhere in the middle of s. Split s at the point into a, b with strictly shorter length. Claim $a, b \in K$ because they're non-empty (✓), $\#open_a = \#closed_a, \#open_s - \#open_a = \#open_b = \#closed_s - \#closed_a = \#closed_b$ (✓) $v \geq 0$ when scanning a, b (✓). By **IH**, $a, b \in K \Rightarrow a, b \in L \Rightarrow ab \in L$ (CONCAT).

□

Chapter 2

Deterministic Finite Automata

2.1 Definitions

- A *deterministic finite automata* (DFA) M is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where
  - Q: non-empty, finite *set of state*
  - Σ: non-empty, finite *alphabet*
  - $\delta : Q \times \Sigma \to Q$: *transition function*
  - $q_0 \in Q$: *start state*
  - $F \subseteq Q$: *set of accepting state*
- Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA and let $w = w_1 w_2 \dots w_n$ be a string over Σ. The *computation path* of M with respect to w is a sequence of states $r_0, r_1, \dots, r_n$ where $r_0 = q_0$ and $\delta(r_{i-1}, w_i) = r_i$ for each $i \in [n]$.
  An *accepting computation path* is such that $r_n \in F$, and a *rejecting computation path* is such that $r_n \notin F$.
- Extended transition function: Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA. The transition $\delta : Q \times \Sigma \to Q$ can be extended to $\delta^* : Q \times \Sigma^* \to Q$ where $\delta^*(q, w)$ is defined as the state we end up in if we state at q and read the string w.
- For a DFA M, let $L(M)$ denote the set of all strings that M accepts. $L(M) = \{w \in \Sigma^* : M \text{ accepts } w\}$. We refer to $L(M)$ as the language *recognized/accepted/decided* by M.
- A language $L \subseteq \Sigma^*$ is called a *regular language* if there is a DFA M such that $L = L(M)$.
- **Theorem** Let $\Sigma = \{0, 1\}$. The language $L = \{0^n 1^n : n \in \mathbb{N}$ is not regular.
  Let $\Sigma = \{a\}$. The language $L = \{a^{2^n} : n \in \mathbb{N}\}$ is not regular.
- **Theorem** Regular languages are closed under union, intersection, difference, concatenation (and star operation).

2.2 Exercises

1. Is it true that if $L \subseteq \Sigma^*$ is a regular language, then any $L' \subseteq L$ is also a regular language?
   - Counterexample: $L = \Sigma^*$ is a regular language (construct a single-state DFA in which the state is accepting), but $\{0^n 1^n : n \in \mathbb{N}\} \subseteq \Sigma^*$ is not regular.
2. Suppose $L_1, \dots, L_k$ are all regular languages. Is it true that their union $\bigcup_{i=0}^{k} L_i$ must be regular?
   - Yes, by induction.

   Suppose $L_0, L_1, L_2, \dots$ is an infinite sequence of regular languages. Is it true that their union $\bigcup_{i \geq 0} L_i$ must be a regular language?

- No. Note that any language of cardinality 1 is regular, i.e. singleton $\{w\}$ for any $w \in \Sigma^*$. In particular, for any $n \in \mathbb{N}$ the language $L_n = \{0^n 1^n\}$ of cardinality 1 is regular. But $\bigcup_{n \geq 0} L_n = \{0^n 1^n : n \in \mathbb{N}\}$ is not regular.

3. Suppose $L \subseteq \Sigma^*$ is a regular language. Show that the following languages are also regular:

$$\text{SUFFIXES}(L) = \{x \in \Sigma^* : yx \in L \text{ for some } y \in \Sigma^*\}$$
$$\text{PREFIXES}(L) = \{y \in \Sigma^* : yx \in L \text{ for some } x \in \Sigma^*\}$$

Proof. Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA recognizing L. Define $S \subseteq Q$ to be the set of states that are "reachable" stating from the initial state q_0. More precisely,

$$S = \{s \in Q : \exists y \in \Sigma^* \text{ such that } \delta(q_0, y) = s\}$$

Define a DFA for each $s \in S$ as follows: $M_s = (Q, \Sigma, \delta, s, F)$. We claim that

$$\text{SUFFIXES}(L) = \bigcup_{s \in S} L(M_s)$$

Double containment: If $x \in \text{SUFFIXES}(L)$ then there is some $y \in \Sigma^*$ such that $yx \in L$. So $M(yx)$ accepts. Note that this y, when fed into M, ends up in some state s. Then M_s accepts x, i.e. $x \in L(M)$ and therefore $x \in \bigcup_{s \in S} L(M_s)$. On the other hand suppose $x \in \bigcup_{s \in S} L(M_s)$. Then there is some state $s \in S$ such that $x \in L(M_s)$. By the definition of S, there is some string y such that $M(y)$ ends up in state s. Since x is accepted by M_s, we have that yx is accepted by M, i.e. $yx \in L$. By the definition of SUFFIXES(L), this implies $x \in \text{SUFFIXES}(L)$. This concludes the proof of the claim. Since $L(M_s)$ is regular for all $s \in S$ and S is a finite set, we conclude that SUFFIXES(L) is regular.
For the second part, define the set

$$R = \{r \in Q : \exists x \in \Sigma^* \text{ such that } \delta(r, x) \in F\}$$

Now we can define the DFA $M_R = (Q, \Sigma, \delta, q_0, R)$. Observe that this DFA recognizes PREFIXES(L), which shows that PREFIXES(L) is regular. □

4. (Regular languages are closed under star)

Proof. We first show that if L is regular, then so is $L^+ = \bigcup_{n \in \mathbb{N}^+} L^n$. We construct a DFA $M' = (Q', \Sigma, \delta', q_0', F')$ recognizing L^+ using a DFA $M = (Q, \Sigma, \delta, q_0, F)$ that recognizes L.

- $Q' = \mathcal{P}(Q)$
- For any $S \subseteq Q$ and any $a \in \Sigma$, let

$$\delta'(S, a) = \begin{cases} \{\delta(s, a) : s \in S\} \cup \{q_0\} & \text{if there is } s \in S \text{ such that } \delta(s, a) \in F \\ \{\delta(s, a) : s \in S\} & \text{otherwise} \end{cases}$$

- $q_0' = \{q_0\}$
- $F' = \{S \subseteq Q : S \cap F \neq \varnothing\}$

□

5. Let $D = (Q, \Sigma, \delta, q_0, F)$ be a DFA and let $q \in Q$ be one of its staes. Then there is a string $w \in \Sigma^*$ such that $\delta^*(q_o, w) = q$.

- False. Counterexample: dead state.

6. For languages L, L', if $L \subseteq L'$ and L is irregular, then L' is irregular.

- False. Counterexample: $L = \{0^n 1^n : n \in \mathbb{N}\}, L' = \Sigma^*$.

7. If $L_1, L_2 \subseteq \Sigma^*$ are irregular languages, then so is $L_1 \cup L_2$.

- False. Counterexample: $L_1 = \{0^n 1^n : n \in \mathbb{N}\}, L_2 = \Sigma^* \setminus L_1$ but $L_1 \cup L_2 = \Sigma^*$.

8. Let L be an irregular language. There exists $K \subset L, K \neq L$ such that K is also irregular.

   - True. $L \neq \Sigma^*$ is infinite. Let $K = L \setminus \{w\}$ where $\{w\} \in L$, then $K \neq L$. AFSOC K is regular, then since the singleton $\{w\}$ is regular, $L = K \cup \{w\}$ is regular, contradiction.

9. Show that if $|\Sigma| > 1$, then $L = \{x | x \in \Sigma^* \text{ and } x = x^r\}$ is an irregular language.

   Proof. AFSOC there exists a DFA with k states that decides L. Consider two symbols $a, b \in \Sigma$ (since assuming $|\Sigma| > 1$). Take the strings $b^n a$ for $n \in \{0, \ldots, k\}$. Since there are only k states, by PHP, there must exist some $i, j, 0 \leq i < j \leq k$ such that $b^i a, b^j a$ end in the same state. Thus, $b^i a b^i, b^j a b^i$ must end in the same state. However, $b^i a b^i$ is a palindrome so it should end up in an accepting state, and $b^j b^i$ is not a palindrome so it should end up in a rejecting state. This is a contradiction since a state cannot be both accepting and rejecting. □

10. For any $k \geq 1$, let $R_k = \{x \in \{0,1\}^* \text{ and the } k\text{-th symbol from the right is a } 1\}$. Show that any DFA that accepts R_k must have at least 2^k states.

    Proof. Consider the set B_k of all binary strings of length k. Note that $|B_k| = 2^k$. AFOSC that there are fewer than 2^k states in a DFA M which decides R_k. By PHP, we must have at least two strings in B_k, denoted x, y that end up in the same state of M. Since these are different strings, x, y must have some bit x_i, y_i such that $x_i \neq y_i$. Consider the strings $x' = x0^i, y' = y0^i$. Note that x', y', the bits x_i, y_i are the k-th characters from the right. Since x, y end up in the same state, and we append the same string 0^i to both to obtain x', y', we know x', y' end up in the same state. However, since $x_i \neq y_i$, only one of x', y' should end up in an accepting state. This is a contradiction. □

11. Let $\Sigma = \{0, 1\}$. Define $C_3 = \{x \in \Sigma^* | x \text{ is a binary number that is a multiple of } 3\}$. Show that C_3 is regular.

    Proof. Let $Q = \{q_0, q_1, q_2\}$ corresponding to the possible remainders modulo 3. We want the following property to hold: a string corresponding to the binary representation of a number w should end on state q_i if $w \equiv i \mod 3$. Set $F = \{q_0\}$.
    As we read a binary string s, we can determine what number/value it corresponds to as follows: State with value 0. Scan s from left to right. If we see a 0, multiply the value by 2. If we see a 1, multiply the value by 2 and add 1. With this observation, we'll define our transition function as $\delta(q_i, 0) = q_{2i}, \delta(q_i, 1) = q_{2i+1}$, where indices of the q's are taken modulo 3. □

Chapter 3

Turing Machines

3.1 Definitions

- A *Turing machine* (TM) M is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ where
  - Q: a non-empty finite *set of state*
  - Σ: a non-empty finite set of *input alphabet* that doesn't contain the *blank symbol* $\sqcup$
  - Γ: a finite set of *tape alphabet* such that $\sqcup \in \Gamma, \Sigma \subset \Gamma$
  - $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$: *transition function* (no transition out of accepting and rejecting states)
  - $q_0 \in Q$: *initial state*
  - $q_{\text{acc}} \in Q$: *accepting state*
  - $q_{\text{rej}} \in Q$: *rejecting state*
- A Turing machine uses a *tape*, a two-direction infinite sequence of *cells*, as memory.
  A *tape head* initially points to index 0.
- A *configuration* for M is defined to be a string $uqv \in (\Gamma \cup Q)^*$ where $u, v \in \Gamma^*, q \in Q$:
  (i) the state M is in: q
  (ii) the contents of the tape: $\cdots \sqcup \sqcup \sqcup uv \sqcup \sqcup \sqcup \ldots$
  (iii) position of the tape head: the leftmost symbol of v

  A configuration is an *accepting configuration* if $q = q_{\text{acc}}$ and a *rejecting configuration* if $q = q_{\text{rej}}$.
  Suppose M reaches a configuration α (not accepting or rejecting). Knowing just the configuration and δ, one can determine the configuration β that M will reach at the next step of computation: $\alpha \vdash_M \beta$ (α yields β in M).
  Given an input $x \in \Sigma^*$, $M(x)$ *halts* if there exists a sequence of configuration (called the *computation trace*) $\alpha_0, \alpha_1, \ldots, \alpha_T$ such that
  - $\alpha_0 = q_0 x$
  - $\alpha_t \vdash \alpha_{t+1}$ for all $t = 0, 1, \ldots, T-1$
  - α_T is either an accepting or rejecting configuration

  Otherwise, $M(x)$ *loops*.
- A Turing machine is called a *decider* if it halts on all inputs.
  Let M be a Turing machine (not necessarily a decider). We denote by $L(M)$ the set of all strings that M *accepts*.
  When M is a decider, then M *decides* the language $L(M)$.
  A language L is called *decidable/computable* if $L = L(M)$ for some decider Turing machine M.

- The Church-Turing Thesis (CTT) states that <u>any computation that can be conducted in this universe constrained by the laws of physics can be carried out by a TM</u>.
  (i) CTT says nothing about the efficiency of the simulation.
  (ii) CTT is a physical claim about the universe we live in.
  (iii) CTT implies that any computation that can be carried out by a human can be carried out by a TM.
- Let Σ be some finite alphabet. A *universal Turing machine* U is a Turing machine that makes $\langle M, x\rangle$ as input, where M is a TM and x is a word in Σ^*:

```
U(<M, x>):
  Simulate M on input x.
  If it accepts, accept.
  If it rejects, reject.

```

  To make sure M always halts, we can add an integer k as input and have U simulate for at most k steps.
- Fix some alphabet Σ, the followng languages are decidable:

$$\begin{aligned}
&\text{ACCEPTS}_{\text{DFA}} = \{\langle D, x\rangle : D \text{ is a DFA that accepts the string } x\}\\
&\text{SELF-ACCEPTS}_{\text{DFA}} = \{\langle D\rangle : D \text{ is a DFA that accepts the string } \langle D\rangle\}\\
&\text{EMPTY}_{\text{DFA}} = \{\langle D\rangle : D \text{ is a DFA with } L(D) = \varnothing\}\\
&\text{EQ}_{\text{DFA}} = \{\langle D_1, D_2\rangle : D_1, D_2 \text{ are DFAs with } L(D_1) = L(D_2)\}
\end{aligned}$$

3.2 Exercises

1. In the 7-tuple definition of a TM, the tape does not appear. Why is this the case?
   - The transition function is a part of the 7-tuple and defines the TM's interactions with the tape.
2. In the definition of a TM, the number of states is restricted to be finite. What is the reason for this restriction?
   - To make TM realistic enough to model real-world computation.
3. State 4 differences between TMs and DFAs.
   - 1. TM has but DFA doesn't have an infinite tape; 2. TM can but DFA cannot go back and read input repeatedly; 3. TM can loop but DFA either accepts or rejects; 4. TM has only one accepting state but DFA can have more.
4. Let $D = (Q, \Sigma, \delta, q_0, F)$ be a DFA. Define a decider TM M such that $L(M) = L(D)$.
   - Add new accept and reject states. Use the DFA transition function, move right every time, write anything to the tape. For every state, add a transition that reads the blank symbol to the accept/reject state, depending on whether that state is accepting or rejecting in DFA.
5. Prove $\text{INF}_{\text{DFA}} = \{\langle D\rangle : D \text{ is a DFA with } L(D) \text{ infinite}\}$

   Solution. We first prove if a DFA with k states accepts some string w with $|w| > k$, then it will accept infinitely many strings. Let D be a DFA with k states accepting the string $w = w_1 w_2 \dots w_n, n > k$. By PHP, we must have $w_1 \dots w_i$ and $w_1 \dots w_j, i < j$ end on the same state $q \in Q$. It follows that $w_1 \dots w_i (w_{i+1} \dots w_j)^m$ ends on q for all $m \in \mathbb{N}$. So all the strings of the form $w_1 \dots w_i (w_{i+1} \dots w_j)^m w_{j+1} \dots w_n$ for $m \in \mathbb{N}$ are accepted.
   We construct a decider M for INF_{DFA} as follows:

```
Given input <D> where D is a DFA, let k be the number of states of D
Construct a DFA D' such that L(D')={w ∈ Σ*: |w| > k}
Construct a DFA D'' such that L(D'') = L(D') ∩ L(D)
Let M_EMPTY-DFA be a decider for EMPTY_DFA and run M_EMPTY-DFA on <D''>
If M_EMPTY-DFA accepts, reject. Else accept.

```

Proof of correctness. Suppose $\langle D \rangle \in \text{INF}_{\text{DFA}}$. This means $L(D)$ is infinite. Since there are only finitely many strings of length at most k, D accepts some string of length greater than k. So $L(D'')$ is nonempty. This means $M_{\text{EMPTY-DFA}}(\langle D'' \rangle)$ rejects, which means $M(\langle D \rangle)$ accepts as desired.
Suppose $\langle D \rangle \notin \text{INF}_{\text{DFA}}$. This means $L(D)$ is finite. Then by the contrapositive of our lemma it accepts no strings of length greater than k. It follows that $L(D'') = \varnothing$, so $M_{\text{EMPTY-DFA}}(\langle D'' \rangle)$ accepts and $M(\langle D \rangle)$ rejects as desired. ■

6. Suppose we change the definition of a TM so that the transition function has the form $\delta : Q \times \Gamma \to Q \times \Gamma \times \{R, S\}$ where the meaning of S is "stay". That is, at each step, the tape head can move one cell to the right or stay in the same position. Suppose $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$ is a TM of this new kind, and suppose also that M is a decider. Show that $L(M)$ is a regular language by describing a DFA recognizing $L(M)$.

   Solution. First given a TM, we can remove all the dead states that can never be reached, so AWLOG M doesn't have dead states. We construct a DFA $(Q', \Sigma, \delta', q_0', F')$ that recognizes the language $L(M)$:

   - $Q' = Q, q_0' = q_0$
   - For the transition function $\delta' : Q' \times \Sigma \to Q'$,
     - Case 1: If (q, a) is such that $q \in \{q_{\text{acc}}, q_{\text{rej}}\}$, then define $\delta'(q, a) = q$. So the $q_{\text{acc}}, q_{\text{rej}}$ states are sink states.
     - Case 2: If we're not in case 1, then we consider whether the transition $\delta(q, a)$ moves the tape head to the right or stays. In the case that the head moves right, if $\delta(q, a) = (q', b, R)$ then define $\delta'(q, a) = q'$
     - Case 3: If $\delta(q, a) = (q', b, S)$, then either the machine reaches an accepting or rejecting state before ever moving the tape head right, or it will eventually move the tape head right and enter a state q''. If the machine reaches an accepting state, define $\delta'(q, a) = q_{\text{acc}}$. if it reaches a rejecting state, define $\delta'(q, a) = q_{\text{rej}}$. Otherwise, define $\delta'(q, a) = q''$.
   - We have $q_{\text{acc}} \in F'$, but F' may contain other states as well. In particular, let's call a state q *good* if running M on the blank input, starting at q, end at q_{acc}. Then we put all the good states in F'. ■

7. Fix an alphabet Σ. We say that a language $L \subseteq \Sigma^*$ is *semi-decidable/recognizable* if there is a TM M such that $L = L(M)$. Show that L is decidable iff $L, \bar{L} = \Sigma \setminus L$ are semi-decidable.

   Proof.
   - ($\Rightarrow$) Note that if a language is decidable, then it is automatically semi-decidable. Therefore, observing that decidable languages re closed under complementation: if we take a decider M for L and reverse the accepting and rejecting states, then we'll have a decider for $\bar{L}$, we have both $L, \bar{L}$ semi-decidable.
   - ($\Leftarrow$) Let M be a semi-decider for L and let M' be a semi-decider for $\bar{L}$. We construct a decider TM N for L as following:

```
def N(x):
  for t = 1, 2, 3, ...:
    simulate M(x) for t steps
    if it halts and accepts, accept
    simulate M'(x) for t steps
    if it halts and accepts, reject

```

   Proof of correctness. If $x \in L$, then we know that since M is a semi-decider for L, $M(x)$ accepts after some number of steps. We also know that M' only accepts strings that are not in L, and therefore does not accept x. So $N(x)$ will correctly accept.
   If $x \notin L$, then we know that $x \in \bar{L}$. Since M' is a semi-decider for $\bar{L}$, $M'(x)$ accepts after some number of steps. We also know that M only accepts strings in L, and therefore does not accept x. $N(x)$ will correctly reject.

   □

Chapter 4

Countable and Uncountable Sets

4.1 Definitions

- Let A, B be two (possibly infinite) sets.
  - $f : A \to B$ is *injective*: $\forall a, a' \in A, f(a) = f(a') \Rightarrow a = a'$. Notation: $A \hookrightarrow B$.
  - $f : A \to B$ is *surjective*: $\forall b \in B, \exists a \in A, f(a) = b$. Notation: $A \twoheadrightarrow B$.
  - $f : A \to B$ is *bijective* (*one-to-one Correspondence*): both injective and surjective. Notation: $A \leftrightarrow B$.
- Let A, B, C be three (possibly infinite) sets. Then
  1. $A \hookrightarrow B \Leftrightarrow B \twoheadrightarrow A$
  2. $A \hookrightarrow B, B \hookrightarrow \Rightarrow A \hookrightarrow C$
  3. $A \leftrightarrow B \Leftrightarrow A \hookrightarrow B$ and $B \hookrightarrow A$
- If $A \leftrightarrow B$, then $|A| = |B|$. If $A \hookrightarrow B$ or $B \twoheadrightarrow A$, then $|A| \leq |B|$. $|A| < |B|$ if it is not the case that $|A| \geq |B|$.
- A is called *countable* if $|A| \leq |\mathbb{N}|$; A is *countably infinite* if it is countable and infinite; A is *uncountable* if $|A| > |\mathbb{N}|$. A is countably infinite iff $|A| = |\mathbb{N}|$.
- $\mathbb{Z} \times \mathbb{Z}, \mathbb{Q}, \Sigma^*$ where Σ, the set of all polynomials in one variable with rational coefficients is finite is countable. CS method: there is an encoding (injective function) Enc: $A \to \Sigma^*$ for some finite alphabet Σ.
- (Cantor's Theorem): For any set A, $|\mathcal{P}(A)| > |A|$.
- Let Σ be some finite alphabet. Σ^∞ is the set of all infinite length words over the alphabet Σ. $\Sigma^* \cap \Sigma^\infty = \varnothing$.
- If A is uncountable and $A \subseteq B$, then B is uncountable.
- The set of languages, $\mathcal{P}(\mathbb{N}), \{0,1\}^\infty$, all bijective functions $\mathbb{N} \to \mathbb{N}$ is uncountable.

4.2 Exercises

1. Show that the following sets are uncountable.
   (a) The set of all bijective function $\mathbb{N} \to \mathbb{N}$.
   (b) $\{x_1 x_2 x_3 \cdots \in \{1,2\}^\infty : \text{for all } n \geq 1, \sum_{i=1}^n x_i \not\equiv 0 \mod 4\}$

   Proof. Part 1: Let A be the set of all bijective functions $\mathbb{N} \to \mathbb{N}$. We want to show that A is uncountable by showing that $\{0,1\}^\infty \hookrightarrow A$, establishing $\{0,1\}^\infty| \leq |A|$.
   Given $x \in \{0,1\}^\infty$, let x_n be the nth bit of x, we map it to a bijection $f_x : \mathbb{N} \to \mathbb{N}$ via

$$\text{if } x_n = 0,\ f_x \text{ maps } 2n \text{ to } 2n \text{ and } 2n+1 \text{ to } 2n+1$$
$$\text{if } x_n = 1,\ f_x \text{ maps } 2n \text{ to } 2n+1 \text{ and } 2n+1 \text{ to } 2n$$

Observe that for any $x \in \{0,1\}^\infty$, the corresponding function f_x is indeed a bijection. it is also clear that $f_x = f_{x'} \Rightarrow x = x'$. So the mapping is injective. This completes the proof.
Part 2: Let $A = \{x_1x_2x_3\cdots \in \{1,2\}^\infty : \text{for all } n \geq 1, \sum_{i=1}^n x_i \not\equiv 0 \mod 4\}$. We want tos how that A is uncountable by identifying a subset of A that is in one-to-one Correspondence with $\{0,1\}^\infty$.
Let $a = 22, b = 112$. Define the set $A' = \{1w : w \in \{a,b\}^\infty\}$. Note

- $A' \subseteq A$
- The mapping $f : \{0,1\}^\infty \to \{a,b\}^\infty$ such that $y_1y_2\cdots \in \{0,1\}^\infty$ is mapped to $w_1w_2\cdots \in \{a,b\}^\infty$ where $w_i = a$ if $y_i = 0$ adn $w_i = b$ if $y_i = 1$ is a bijection.

These two observations imply that we have a subset of A uncountable, so A is uncountable. □

2. Let S be a subset of the positive reals with the property that for every real number x, the set $\{y \in S : y > x\}$ has a minimum element. Determine whether S is necessarily countable.

Solution. The set is countable. We'll use the fact that rationals are dense in the reals: inject S to $\mathbb{Q}$ by mapping each element $s \in S$ to some rational between x and the next largest element of S. ■

Chapter 5

Undecidable Languages

5.1 Definitions

- Fix some input alphabet and tape alphabet. The set of all Turing machines $\{M : M \text{ is a TM}\}$ is countable.
- Almost all languages are undecidable.
- *Halting problem*: HALTS= $\{\langle M, x\rangle : M \text{ is a TM which halts on input } x\}$ is undecidable.

```
M_TURING(<M>):
  Run M_HALTS(<M, M>)
  If it accepts, go into an infinite loop
  If it rejects, accept

```

  If $M_{\text{HALTS}}(\langle M_{\text{TURING}}, M_{\text{TURING}}\rangle)$ accepts, then $M_{\text{TURING}}(\langle M_{\text{TURING}}\rangle)$ is supposed to halt by definition of M_{HALTS}. However, from the description of M_{TURING} above, it goes into an infintie loop, contradiction. If $M_{\text{HALTS}}(\langle M_{\text{TURING}}, M_{\text{TURING}}\rangle)$ is supposed to lead to an infinite loop. But from the description, it accepts, and therefore halts, contradiction.

- The languages ACCEPTS= $\{\langle M, x\rangle : M \text{ is a TM that accepts the input } x\}$, EMPTY= $\{\langle M\rangle : M \text{ is a TM with } L(M) = \varnothing\}$, EQ= $\{\langle M_1, M_2\rangle : M_1, M_2 \text{ are TMs with } L(M_1) = L(M_2)\}$ are undecidable.
- Let A, B be two languages. *A reduces* to $B, A \leq B$: assume B is decidable and then show that A is decidable by using the decider for B as a subroutine. If$A \leq B$ and A is undecidable, then B is also undecidable.

5.2 Exercises

1. Given the encoding of a DFA D, we can determine if D accepts a string by checking if in the state diagram there is a directed path from the initial state to one of the accepting states. Show that the strategy doesn't apply to TMs by drawing the state diagram with $L(M) = \varnothing$ but there is a path from the starting state to the accepting state.
2. Let FINITE= $\{\langle M\rangle : M \text{ is a TM and } L(M) \text{ is finite}\}$, TOTAL= $\{\langle M \text{ is a TM which halts on all inputs}\}$. Show that TOTAL $\leq$ FINITE.

   Solution. Suppose that M_{FINITE} decides FINITE. We define a decider for TOTAL:

```
def M_TOTAL(<M>):
  <HELP> =
    """def HELP(w):
        for y <= w:
            run M(y)
        ACCEPT"""
    return not M_FINITE(<HELP>)

```

Suppose the input string is $\langle M \rangle$ such that M halts on all inputs. Then notice that $L(HELP) = \Sigma^*$. Therefore $M_{\text{FINITE}}(\langle HELP \rangle)$ rejects, which means M_{TOTAL} accepts as desired.

Suppose the input string is $\langle M \rangle$ such that M does not halt on all inputs. Let x be the lexicographically smallest string that M does not halt on. Then HELP will not accept any string lexicographically greater or equal to than x, as $M(x)$ will not halt. Since there are only finitely many strings lexicographically smaller than x, $L(HELP)$ is finite. Thus, $M_{\text{FINITE}}(\langle HELP \rangle)$ accepts, which means M_{TOTAL} rejects as desired. ■

Chapter 6

Time Complexity

6.1 Definitions

- (Big-O) For $f : \mathbb{R}^+ \to \mathbb{R}^+$, we write $f(n) = O(g(n))$ if there exist constants $C > 0$ and $n_0 > 0$ such that for all $n \geq n_0$, $f(n \leq Cg(n))$.
  (Big-Omega) For $f : \mathbb{R}^+ \to \mathbb{R}^+$, we write $f(n) = \Omega(g(n))$ if there exist constants $c > 0$ and $n_0 > 0$ such that for all $n \geq n_0$, $f(n \geq cg(n))$.
  (Theta) For $f : \mathbb{R}^+ \to \mathbb{R}^+, g : \mathbb{R}^+ \to \mathbb{R}^+$, $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$. This is equivalent to saying that there exists constants $c, C, n_0 > 0$ such that for all $n \geq n_0, cg(n) \leq f(n) \leq Cg(n)$.
  The expression of the running time of an algorithm using big-O, big-Omega or Theta notation is referred to as *asymptotic complexity* estimate of the algorithm.

- The *worst-case running time* of an algorithm A is a function $T_A : \mathbb{N} \to \mathbb{N}$ defined by $T_A(n) = max_{\text{instances/inputs } x \text{ of length } n}$ number of steps A takens on input x.

- Constant time $O(1)$, logarithmic time $O(\log n)$, linear time $O(n)$, quadratic time $O(n^2)$, polynomial time $O(n^k)$ for some constant $k > 0$, exponential time $O(2^{n^k})$ for some constant $k > 0$.

- The *intrinsic complexity* of a computational problem refers to the asymptotic time complexity of the most efficient algorithm that computes the problem.
  The intrinsic complexity of $L = \{0^k 1^k : k \in \mathbb{N}\}$ is $\Theta(n)$.

- Karatsuba algorithm for integer multiplication solves the integer multiplication problem in time $O(n^{1.59})$.

6.2 Exercises

- Determine which of the following problems can be computed in worst-case polynomial-time.
  - Give an input positive integer N, output $N!$.
    - To represent the value of $N!$, we will need $\log_2(N!) = \Theta(N \log N)$ bits. It takes at least one step to write down a signle bit of the output. Therefore, writing out the whole output would take exponential time with respect to the input length n, since $n = O(\log N)$.
  - Given as input a positive integer N, output True iff $N = M!$ for some positive integer M.
    - The polynomial-time algorithm is as follows. We iteratively compute the values $1!, 2!, 3!, \ldots$. In particular, at iteration M, in order to compute $M!$, we take the result from previous iteration and multiply it by M. In every iteration we check whether $M! < N$. When $M! \geq N$, we exit the loop and check if $M! = N$, then return true, and false otherwise.
      Analysis of algorithmic complexity: first, the number of iteration is upper bounded by $O(\log N)$ because after the second iteration, $M!$ grows by a factor of at least 2 in each iteration. In each iteration we do a

multiplication operation. Using the quadratic-time multiplication algorithm, we ahve the running time of the multiplication operation at most $O(\log^2 N)$. Therefore, the total running time of the loop is $O(\log^3 N)$. The comparison between two numbers $M!, N$ takes $O(\log N)$. Thus, the whole algorithm takes $O(\log^3 N) = O(n^3)$, which is poly-time.

- Given as input a positive integer N, output True iff $N = M^2$ for some positive integer M.
  - We will perform binary search to pin down the value of x such that $x^2 = N$. At each iteration, we have a guess x and compute x^2 and compare it with N.
    Anlaysis of run-time: first, the number of iteration is upper bounded by $O(\log N)$. In terms of the word done in each iteration, the value of x is at most N. Hence, we can do multiplication in $O(\log^2 N)$ time. We also perform comparisons and assignments, each of which only takes $O(\log N)$ time, so the work done in each iteration is $O(\log^2 N)$. We can conclude that the total time is $O(\log^3 N)$, which is polynomial in $n = \log N$.

- Let f, g, h be functions from $\mathbb{N}^+$ to $\mathbb{N}^+$. Prove or disprove the following:
  - If $f = O(g)$ and $g = O(h)$, then $f = O(h)$.
    - True. By definition, there exist n_1, c_1 such that $\forall n \geq n - 1, f(n) \leq c_1 g(n)$ and similarly n_2, c_2 for g and h. Choose $n_0 = \max(n_1, n_2), c = c_1 c_2$ and let $n \geq n_0$. Then $f(n) \leq c_1 g(n) \leq c_1 c_2 h(n) = ch(n)$ as desired.
  - If $f = O(g)$ then $g = O(f)$.
    - Counterexample: let $f(n) = 1, g(n) = n$. While $f = O(g)$, there's no n_0, c you can pick that makes $g = O(f)$ because in order to do this, we neec $g(n) = n \leq c\dot{1} = c\dot{f}(n), \forall n \geq n_0$. However, no matter what constants c, n_0 are chosen, the inequality will not be satisfied when $n > c$.
  - $f = O(g)$ or $f = \Omega(g)$.
    - Counterexample: let $f(n) = 1$ if n is even and $f(n) = n$ if n is odd. Let $g(n) = n$ if n is even and $g(n) = 1$ if n is odd. By the same argument as the previous problem, if we attempt to choose c, n_0 for either O or Ω, we will be able to find a sufficiently large n (even or odd as necessary) to disprove the inequality.

- Name a function $f(n)$ which is asymptotically super-polylogarithmic, i.e. $f(n) = \Omega(\log^c n)$ for any constant $c > 1$, and at the same time asymptotically sub-polynomial, i.e. $f(n) = O(n^\epsilon)$ for any constant $\epsilon > 0$.
  - $f(n) = 2^{\sqrt{\log n}}$.

Chapter 7

Special Topic: Combinatorial Games

7.1 Definitions

1. **Theorem ($\mathbf{TAG_3}$)** Suppose we start off with n chips. Then the first player has a winning strategy for all n that is not a multiple of 4. For n being a multiple of 4, the second player can always win, regardless of what strategy the first player plays.

   Proof. The proof is by induction. Start with $n = 0$, then the first player loses. For $n = 1, 2, 3$ the first player wins. Now suppose the theorem is true for all $k < n$. And suppose $n = 4a + b$ for $a, b \in \mathbb{Z}$ where $a \geq 1, b \in [3]$. If $b \neq 0$, then consider the move that removes b chips. This results in $4a$ chips on the table, so by the **IH**, the next player will lose. Hence we've given a winning strategy for the first player when n is not a multiple of 4 - pick up $n \mod 4$ chips.
   If $b = 0$, then no matter how many chips $c \in [3]$ the first player picks up, the second player can remove $4 - c$ chips and return to a smaller multiple of 4, which by **IH**, is a loss for the first player. This proves the recursive case. □

2. A *combinatorial game* is a game involving two players which is in some *state* at any point in time, and this state is known to both players (*perfect information*). In any state, each of the two players has a set of *valid moves*, and each of these valid moves causes the state to change to some other well-defined state.
   The game starts in some *initial state* and the players alternate moving. The game has reached a *terminal state* when there are no valid moves for the player whose turn it is.

3. *Normal rule*: the last player to play wins.
   Misère rules: the person who plays the last move loses.

4. *Impartial game*: in any state, the set of valid moves are the same, regardless of whose turn it is next, as opposed to *partizan games*.

5. Combinatorial games are games of perfect information (no battleship), without any element of chance (no dice), without draws (no tic-tac-toe).

6. An *N-position* is one where the player to play next has a winning strategy and a *P-position* is one where the player to play next has no winning strategy.

7. Formal inductive definition:

   - All terminal positions are P-positions.
   - A position such that all moves from this position lead to N-position is a P-position.
   - A position such that there exists at least one move from this position leading to a P-position is an N-position.

   A N-position is one where at least one move leads to a P-position.
   A P-position is one from which there are no moves leading to an P-position - all of them lead to N-positions.

8. A position is a win for the first player if and only if it is an N-position.

9. **Theorem (Chomp)** If $n > 1$, the $n \times n$-square position is an N-position.

   Proof. To show this, it suffices to show a strategy for the first player: indeed, she should play (2,2) as the first move. Then we're left with an L-shaped board with equal length arms, which we claim is a P-position. Indeed, if the arms of the L-shaped piece have non-zero length, whatever move the second player makes (which is either $(1, y)$ or $(x, 1)$), the first player can then play the "mirror-image" of that move (namely either $(y, 1)$ or $(1, x)$ respectively) to get back to an L-shaped board with arms of equal length. Eventually both arms will be depleted, and the second player will lose. In other words, the L-shaped position with arms of equal length is a P-position. And hence the square board is an N-position. □

10. *Mirroring*: show that whatever the next player does, you can always make another valid move to prevent him from winning.

11. **Theorem 4.2 (Chomp)** The $m \times n$-rectangular position is an N-position as long as not both of n or m equal 1.

    Proof. Since at least one of $m, n > 1$, consider the move (m, n). Either this is a P-position, in which case the rectangular board was an N-position, or this is an N-position, so there must be some move (x, y) that leads to a P-position. But then we could have made the move (x, y) right from the initial rectangular position to get to the same P-position. So the original rectangular board is an N-position in this case too. □

    Strategy-stealing: we don't know what the winning first move is, but we know one exists, so either our first move was a winning one, or the other player now has a winning move and we can steal it.

12. A single pile nim is an N-position.
    In two-pile nim, we have a P-position if and only if both piles are equal (winning strategy: mirroring).

13. *Nim-sum*: for naturals x, y, define their nim-sum $x \oplus y$ to be their *sum modulo 2 without carry*, or bitwise *exclusive-or* of their binary representation.

    - commutative: $x \oplus y = y \oplus x$
    - associative: $(x \oplus y) \oplus z = x \oplus (y \oplus z)$
    - $x \oplus y \Leftrightarrow x = y$
    - $x \oplus y = x \oplus z \Rightarrow y = z$

14. **Bouton's Theorem** The position (x, y, z) is a P-position iff $x \oplus y \oplus z = 0$.

    Proof. Define Z to be the set of positions (x, y, z) where $x \oplus y \oplus z = 0$ and NZ be the other positions. We will show that Z and NZ satisfy the three properties of being the set of P-positions and the set of N-positions respectively.

    (a) *All terminal positions are P-positions.* Indeed, the only terminal position is $(0, 0, 0)$, which is in Z.

    (b) *All moves from a P-position lead only to N-positions.* Consider any state (x, y, z) in Z - so $x \oplus y \oplus z = 0$. Note that removing tokens from any of the piles (say $x \to x'$) gives us $x' \oplus y \oplus z \neq 0 = x \oplus y \oplus z$, by the cancellation property and the fact that $x \neq x'$. So all moves from $(x, y, z) \in Z$ lead to positions in NZ.

    (c) *There exists a move from an N-position to some P-position.* Consider a position $(x, y, z) \in NZ$: hence $x \oplus y \oplus z = w \neq 0$. We claim we can choose a pile and some number of chips to remove from it to get to a position in Z. Since $w \neq 0$, let the ith bit be the highest bit set to 1 in w. Since w is the bit-wise XOR of x, y, z, at least one of these three must have their i^{th} bit set to 1 - AWLOG it's x.
    Now consider $x' = x \oplus w$. We claim this is the binary representation of a number strictly less than x, because

    $$x - x' = \sum_{j \leq i : x_j = 1, x'_j = 0} 2^j - \sum_{j \leq i : x_j = 0, x'_j = 1} 2^j$$

    But since the first sum contains at least the index i, this is at least $2^i - \sum_{j < i : x_j \neq x'_j} 2^j \geq 1$. Hence we can remove $x - x'$ chips from the pile x to get to (x', y, z). Moreover, $x' \oplus y \oplus z = (x \oplus w) \oplus y \oplus z = (x \oplus y \oplus z) \oplus w = w \oplus w = 0$, so we have shown how to get to some position in Z.

□

15. *Sum of games:* A, B are games. Game $A + B$ is a new game where the allowed moves are to pick one of the two games A or B (that is in a non-terminal position) and make a legal move in that game.
    Terminal positions of $A + B \equiv$ terminal positions in both A, B.
    The sum operator is commutative $A + B = B + A$ and associative $(A + B) + C = A + (B + C)$.

16. The *MEX* of a finite set of natural numbers is the minimum excluded element.

17. *Nimber* ("Sprague-Grundy number") is a number assigned to positions of any (normal, impartial) game.

    - $N(G) = 0$ if G is terminal
    - $N(G) = MEX\{N(G_1), N(G_2), \ldots, N(G_n)\}$ where $G_1, G_2, \ldots, G_n$ are the successor positions of G (i.e. the positions resulting from all the allowed moves)

    Key Observation: Given G, next player can move to a position G' with any desired value of $N(G') < N(G)$.

18. **Theorem** Nimber of a position with a single pile of k chips, $N(P_k) = k$.

19. **Theorem** Any game/position G is a P-position iff $N(G) = 0$ (for any game).

20. **The Nimber Theorem** Let A and B be two impartial normal games. Then $N(A + B) = N(A) \oplus N(B)$.

Chapter 8

Introduction to Graph Theory

8.1 Definitions

- An *undirected graph* G is a pair (V, E) where V is a finite non-empty set called the set of *vertices / nodes* and E is a set called the set of *edges* and every element of E is of the form $\{u, v\}$ for distinct $u, v \in V$.

  Two representation: *adjacency matrix* for a graph of order n is an $n \times n$ matrix A such that $A[i,j] = \begin{cases} 1 & \text{if } \{v_i, v_j\} \in E \\ 0 & \text{otherwise} \end{cases}$

  (nonideal for sparse graph) and *adjacency list*.

- **Handshake Theorem**: Let $G = (V, E)$ be a graph, then $\sum_{v \in V} d(v) = 2m$.

- **Theorem**: Let $G = (V, E)$ be a connected graph with n vertices and m edges. Then $m \geq n-1$. Furthermore, $m = n-1$ iff G is acyclic.

- A graph satisfying two of the following three properties is called a *tree*: connected, $m = n - 1$, acyclic.
  A vertex of degree 1 in a tree is called a *leaf*, and a vertex of degree more than 1 is called an *internal node*.
  A tree has at least 2 leaves.

- A *directed graph* G is a pair (V, A) where V is a non-empty finite set called the set of *vertices / nodes* and A is a finite set called the set of *directed edges / arcs* and every element of A is a tuple (u, v) for $u, v \in V$. If $(u, v) \in A$, we say that there is a directed edge from u to v. Note that $(u, v) \neq (v, u)$ unless $u = v$.

- Let $G = (V, A)$ be a directed graph. For $u \in V$, we define the neighborhood of $u, N(u) = \{v \in V : (u, v) \in A\}$. The *out-degree* of u, $d_{out}(u) = |N(u)|$. A vertex with out-degree 0 is called a *sink*, in-degree 0 a *source*.

- Graph Algorithms:

  - Graph searching algorithms:

    * Arbitrary-first (AFS):

```
G = (V,E): graph. s: vertex in V.
AFS(<G, s>):
  Put (⊥, s) into bag.
  While bag is non-empty:
    Pick an arbitrary tuple (p, v) from bag.
    If v is unmarked:
      Mark v.
      parent(v) = p
      For each neighbor w of v:
        Put (v, w) into bag.

AFS2(<G>):
  For v not marked as visited:
```

```
            Run AFS(<G,v>).

```

  * Breadth-first search (BFS): AFS with bag chosen as a *queue* data structure: $O(m+n)$ for adjacency list
  * Depth-first search (DFS): AFS with bag chosen as a *stack* data structure: $O(m+n)$ for adjacency list

- MST: in the *minimum spanning tree problem*, the input is a connected undirected graph $G = (V, E)$ together with a *cost function* $c : E \to \mathbb{R}^+$. The output is a subset of the edges of minimum total cost such that, in the graph restricted to these edges, all the vertices of G are connected. For convenience, we'll assume the edges have unique edge costs.

  Unique edges costs imply unique MST.

  Theorem (MST cut property): Suppose we are given an instance of the MST problem. For any $V' \subset V$, let $e = \{u, w\}$ be the cheapeast edge with the property that $u \in V', w \in V \setminus V'$. Then e must be in the minimum spanning tree.

  Theorem (Jarník-Prim algorithm for MST): Let $G = (V, E)$ and cost function $c : E \to \mathbb{R}^+$: $O(nm)$

```
MST(<G,c>):
  V' = {u} (for some arbitrary u ∈ V)
  E' = ∅
  While V' ≠ V:
    Let {u, v} be the minimum cost edge such that u ∈ V' but v ∉ V'.
    Add {u, v} to E'.
    Add v to V'.
  Output E'.

```

- Topological sorting: A *topological order* of an n-vertex directed graph $G = (V, A)$ is a bijection $f : V \to \{1, 2, \ldots, n\}$ such that if $(u, v) \in A$, then $f(u) < f(v)$.

  Cycle implies no topological order.

  In the *topological sorting problem*, the input is a directed acyclic graph, and the output is a topological order of the graph.

  Lemma (Acyclic directed graph has a sink): If a directed graph is acyclic, then it has a sink vertex.

  Theorem (Topological sort via DFS): $O(n+m)$

```
Top-Sort(<G>):
  p = |V|.
  For v not marked as visited:
    Run DFS'(<G, v>).
      G = (V, A): directed graph. v: v ∈ V.
      DFS'(<G, v>):
        Mark v as "visited".
        For each neighbor u of v:
          If u is not marked visited:
            Run DFS'(<G, u>).
        f(v) = p.
        p = p - 1
  Output f.

```

8.2 Exercises

1. Let T be a tree with L leaves. Let Δ be the largest degree of any vertex in T. Prove that $\Delta \leq L$.

   - *Proof.* Let v be a vertex in the tree such that $d(v) = \Delta$. Consider the graph $T - v$ obtained by deleting v and all the edges incident to it. Since T is a tree, we know that $T - v$ contains Δ connected componentsl let us denote them $T_1, \ldots, T_\Delta$. Since T is acyclic, each of the T_i's are also acyclic. Since each T_i is connected and acyclic, each one is a tree. There are two possibilities for each T_i:

1. T_i consists of a single vertex. Then that vertex is a leaf in T.
2. T_i is not a single vertex, and so has at least 2 leaves. At least one of these leaves is not connected to v and therefore must be a leaf in T.

In either case, one vertex in T_i is a leaf in T, for all $T_1, \ldots, T_\Delta$. Thus, we have at least Δ leaves in T. $\square$

- Suppose a graph has 2 edges with the same cost. Then there are at least 2 minimum spanning trees of the graph.
  - False. Counterexample: tree.

Chapter 9

Matchings in Graphs

9.1 Definitions

- A *matching* in a graph $G = (V, E)$ is a subset of the edges that do not share an endpoint.
  A *maximum matching* in G is a matching with the maximum number of edges among all possible matchings.
  A *maximal matching* is a matching that if we add any other edge to the matching, it is no longer a matching.
  A *perfect matching* is a matching that covers all the vertices of the graph.
  The size of a matching M refers to the number of edges in the matching, denoted by $|M|$.

- *Maximum matching problem*: the input is an undirected graph $G = (V, E)$, the output is a maximum matching in G.

- Let $G = (V, E)$ be a graph and let $M \subseteq E$ be a matching in G. An *augmenting path* in G with respect to M is a path such that

  1. the path is an *alternating path* whose path alternate between being in M and not in M,
  2. the first and last vertices in the path are not a part of the matching M.

  A single edge $\{u, v\}$ where u, v are not matched is an augmenting path.

- **Theorem**: Let $G = (V, E)$ be a graph. A matching $M \subsetneq E$ is maximum iff there is no augmenting path in G with respect to M.

  Proof. ($\Rightarrow$) Suppose there's an augmenting path $v_1, \ldots, v_k$ in G with respect to M. Since v_1, v_k are not matched and the path is alternating, the number of edges on this path that are in the matching is one less than the number o edges not in the matching. Thus, we can obtain a bigger matching by flipping the matched and unmatched edges on teh path, so M is not maximum.
  ($\Leftarrow$) Let M^* denote a maximum matching in G. Since M is not maximum, we know that $|M| < |M^*|$. We define the set S to be the set of edges contained in M^* or M, i.e. $S = (M^* \cup M) \setminus (M^* \cap M)$. First, note that each vertex that is a part of S has degree 1 or 2 because it can be incident to at most one edge in M and M^*. We make two claims:

  1. each connected component in S is either a path or a cycle;
  2. the edges in these paths and cycles alternate.

  The second claim is true because AFSOC the edges were not alternating, then two edges of the same color share an endpoint. Since M^* is a bigger matching than M, we know that S has more M^* edges than M edges. Observe that the cycles must have even length, otherwise the edges cannot alternate. Then there must be a path with more M^* edges which starts and ends with a M^* edge. Since this path alternate and the endpoints are unmatched with respect to M, so it's an augmenting path as desired. $\square$

- A graph $G = (V, E)$ is called *bipartite* if there is a partition of V into sets X, Y such that all the edges in E have one endpoint in X and the other in Y, denoted as $G = (X, Y, E)$.

Let $G = (V, E)$ be a graph. Let $k \in \mathbb{N}^+$. A *k-coloring* of V is just a map $\chi : V \to C$ where C is a set of cardinality k. I k-coloring is said to be *legal* for G if every edge in E is *bichromatic*. G is k-colorable if it has a legal k-coloring.

- **Theorem**: A graph is bipartite iff it contains no odd-length cycles.
- **Theorem**: There is a polynomial time algorithm to solve the maximum matching problem in bipartite graphs.

  Proof. Let $G = (X, Y, E)$ be the input graph. The algorithm is as follows:

  - Let $M = \{\{x, y\}\}$ where $\{x, y\} \in E$ is an arbitrary edge.
  - Repeat until there is no augmenting path with respect to M:
    * Find an augmenting path with respect to M.
    * Update M according to the augmenting path.

  Every time we find an augmenting path, we increase the size of our matching by one. When there are no more augmenting paths, we stop and correctly output a maximum matching. If finding an augmenting path can be done in polynomial time, then the overall running time of the algorithm is polynomial time since the loop repeats $O(n)$ times and the work done in each iteration is polynomial time. Given $G = (X, Y, E)$ and $M \subsetneq E$, the algorithm is as follows:

  * Direct edges in $E \setminus M$ from X to Y;
  * Direct edges in M from Y to X;
  * For each unmatched $x \in X$:
    · Run DFS(G, x).
    · If you hit an unmatched $y \in Y$, output the path from x to y.
  * Output "no augmenting path found."

  Observe that there is an augmenting path with respect to M iff there is a directed path (in the modified graph) from an unmatched vertex $x \in X$ to an unmatched vertex $y \in Y$. The running time is polynomial time since the loop repeats at most $O(n)$ times, and the work done in each iteration is polynomial time. □

- **Theorem (Hall's Theorem)**: Let $G = (X, Y, E)$ be a bipartite graph. For a subset S of the vertices, let $N(S) = \bigcup_{v \in S} N(v)$. Then G has a matching covering all the vertices in X iff $\forall S \subsetneq X, |S| \leq |N(S)|$.
- *Corollary*: Let $G = (X, Y, E)$ be a bipartite graph. Then G has a perfect matching iff $|X| = |Y|$ and for any $S \subseteq X$, we have $|S| \leq |N(S)|$.

9.2 Exercises

1. Show that a tree can have at most one perfect matching.

   - AFSOC a tree has two different matchings M, M'. Let $S = (M \cup M') \setminus (M \cap M')$ be the symmetric difference. Since $M \neq M'$, $|S| > 1$. The set S corresponds to a graph in which each vertex has degree at most 2. So the graph consists of disjoint paths and cycles. But it cannot contain any cycles since trees are acyclic. It also cannot contain a path, because the existence of a degree 1 vertex in S implies that this vertex is not matched by either M or M' and this contradict that fact that M, M' are perfect matchings. So S must be empty, which contradicts $|S| > 1$.

Chapter 10

Stable Matchings

10.1 Definitions

- *Stable matching problem*: a tuple of sets (X, Y) with $|X| = |Y|$, and a *preference list* for each element of X and Y. The output of the stable matching problem is a *stable matching*, which is a subset S of $\{(x, y) : x \in X, y \in Y\}$ with the following properties:
  - The matching is a *perfect matching*;
  - There are no *unstable pairs*. A pair $(x, y), x \in X, y \in Y$ is called *unstable* if $(x, y) \notin S$, but they both prefer each other to the elements they are matched to.
- **Theorem (Gale-Shapley proposal algorithm)**: There is a polynomial time algorithm which, given an instance of the stable matching problem, always returns a stable matching.

  Proof. Gale-Shapley algorithm is as follows:
  While there is a male m in X not matched, do the following:
  - Let m be an arbitrary unmatched man.
  - Let w be the highest ranked woman on m's preference list to whom m has not "proposed" yet.
  - Let m "propose" to w.
  - If w is unmatched or w prefers m over her current partner, match m, w. The previous partner of w if there was any, is now unmatched.

  We need to show:
  1. the number of iterations in the algorithm is at most n^2 where $n = |X| = |Y|$;
  2. the algorithm always outputs a perfect matching;
  3. there are no unstable pairs with respect to this matching.

  Proof of (1): Notice that the number of iterations in the algorithm is equal to the total number of proposals made. No man proposes to a woman more than once, so each man makes at most n proposals. There are n men in total, so the total number of proposals is at most n^2.
  Proof of (2): AFSOC the algorithm doesn't output a perfect matching, i.e. some man m is not matched to any woman. Notice that the man got rejected by all the women he proposed to. Either he got rejected because the woman preferred her current partner, or he got rejected by a woman that he was already matched with. Either way, all the women that m proposed to must have been matched to someone at some point in the algorithm. But once a woman is matched, she never goes back to being unmatched. So all the women must be matched. But since there're equal number of men and women, the only way is if all the men are matched, contradiction.
  Proof of (3): We first make a crucial observation that as the algorithm proceeds, a man can only go down in his

preference list, and a woman can only go up in hers. Now consider any pair $(m, w), m \in X, w \in Y$ and m, w are not matched by the algorithm. We want to show that this pair is not unstable. Let w' be the woman that m is matched to, and let m' be the man that w is matched to. There're two cases to consider:

1. m proposed to w at some point in the algorithm, then w rejected m at some point, w prefers m' over m.
2. m never proposed to w, m prefers w' over w.

In either case (m, w) is stable. □

- We say that $m \in X$ is a *valid partner* of $w \in Y$ if there is some stable matching in which m, w are matched. For $u \in X \cup Y$, we define the *best valid partner* of u, denoted $\text{best}(u)$, to be the highest ranked valid partner of u. Similarly, we define the *worst valid partner* of u, denoted as $\text{worst}(u)$ to be the lowest ranked valid partner of u.
- **Theorem (Gale-Shapley is male optimal)**: The Gale-Shapley algorithm always matches a male $m \in X$ with its best valid partner, i.e. it returns $\{(m, \text{best}(m)) : m \in X\}$.

  Proof. AFSOC at the end of the Gale-Shapley algorithm, there is some man not matched to his best valid partner. This means that some man gets rejected by a valid partner. Consider the first time that this happens in the algorithm. Let m be this man and w be the valid partner that rejects m. Let m' be the man that w is matched to right after rejecting m, so w prefers m' over m. Since w is a valid partner of m, there is some stable matching in which m, w are matched. Let w' be the match of m' in this stable matching. We want to show that (m', w) forms an unstable pair. If m' preferred w' over w, then m' would have to be rejected by w' in the algorithm as m' later gets matched to w. This would mean m' was rejected by a valid partner before m was. Thus, m' prefers w over w'. The proof is complete. □

10.2 Exercises

1. Show that the Gale-Shapley algorithm always matches a female $w \in Y$ with its worst valid partner, i.e. it returns $\{(\text{worst}(w), w) : x \in Y\}$.
   - AFSOC that the Gale-Shapley algorithm returns a matching in which some w is matched to m but m is not the worst valid partner of w. Let m' be the worst valid partner of w. So there is some other stable matching in which m', w are matched. Let w' be the match of m in this stable matching. We claim that in this stable matching (m, w) forms an unstable pair. First, w prefers m over m' because m' is the worst valid partner of w. Second m prefers w over w' because the Gale-Shapley algorithm matches m, w and so w is the best valid partner of m.
2. Consider the following variant of the stable matching problem. The input is a set of n people, where n is even. Each person has a preference list that includes every other person. The goal is to find a stable matching. Give an example to show that a stable matching doesn't always exsit.
   - Consider the example with 4 perple a, b, c, d and the preference lists: $a : (c, b, d), b : (a, c, d), c : (b, a, d), d : (a, c, b)$.

Chapter 11

Polynomial-Time Reductions

11.1 Definitions

- *k-coloring problem* **(kCOL)**: the input is an undirected graph $G = (V, E)$, and the output is True iff the graph is k-colorable: $\{\langle G\rangle : G \text{ is a } k\text{-colorable graph}\}$.

- A subset of the vertices is called a *clique* if there is an edge between any two vertices in the subset. We say that G contains a k-clique if there is a subset of the vertices of size k that forms a clique.
  Clique problem **(CLIQUE)**: the input is an undirected graph $G = (V, E)$ and a number $k \in \mathbb{N}^+$, and the output is True iff the graph contains a k-clique: $\{\langle G, k\rangle : G \text{ is a graph}, k \in \mathbb{N}^+, G \text{ contains a } k\text{-clique}\}$.

- A subset of the vertices is called an *independent set* if there is no edge between any two vertices in the subset.
  Independent set problem **(IS)**: the input is an undirected graph $G = (V, E)$ and a number $k \in \mathbb{N}^+$, and the output is True iff the graph contains an independent set of size k: $\{\langle G, k\rangle : G \text{ is a graph}, k \in \mathbb{N}^+, G \text{ contains an independent set of size } k\}$.

- A *literal* refers to a Boolean variable or its negation. A *clause* is an "OR" of literals. A Boolean formula in *conjunctive normal form* (CNF) is an "AND" of clauses. We say that a Boolean formula is a *satisfiable formula* if there is a 0/1 assignment to the Boolean variables that makes the formula evaluate to 1.
  CNF satisfiability problem **(SAT)**: the input is a CNF formula, and the output is True iff the formula is satisfiable: $\{\langle\varphi\rangle : \varphi \text{ is a satisfiable CNF formula}\}$.

- A Boolean circuit with n-input variables ($n \geq 0$) is a directed acyclic graph in which each node of the graph is called a *gate* and each directed edge is called a *wire*. There're 5 types of gates: AND (fan-in 2), OR (fan-in 2), NOT (fan-in 1), input (of in-degree/fan-in 0) and constant gates (labed 0 or 1 and of in-degree/fan-in 0). For each 0/1 assignment of the input variables, the Boolean circuit produces a one-bit output from the *output gate*. The input-output behavior of the circuit defines a function $f : \{0, 1\}^n \to \{0, 1\}$ and we say that f is *computed by the circuit*.
  A *satisfiable circuit* is such that there is 0/1 assignment to the input gates that makes the circuit output 1.
  Circuit satisfiability problem **(CIRCUIT-SAT)**: the input is a Boolean circuit, adn the output is True iff the circuit is satisfiable: $\{\langle C\rangle : C \text{ is a Boolean circuit that is satisfiable}\}$.

All the problems defined above are poly-time decidable.

- *Cook reduction* (Poly-time Turing reduction): Fix some alphabet Σ. Let A, B be two languages. We say that A *poly-time reduces* to B, written $A \leq^P B$, if there is a poly-time decider for A that uses a decider for B as a black-box subroutine.
  Transitivity of Cook reductions: if $A \leq^P B, B \leq^P C$, then $A \leq^P C$.

- Suppose $A \leq^P B$. If $B \in P$, then $A \in P$ and if $A \notin P$ then $B \notin P$, i.e. B is at least as hard as A with respect to poly-time decidability.

- *Karp reduction* (Poly-time many-one reduction): Let A, B be two languages. Suppose that there is a poly-time computable function / transformation $f : \Sigma^* \to \Sigma^*$ such that $x \in A$ iff $f(x) \in B$. Then we say that there is a *poly-time*

many-one reduction from A to B and denote it by $A \leq_m^P B$.
If there is a many-one reduction from language A to language B, then one can construct a regular (Turing) reduction from A to B. Not all reductions are many-one reductions.
Steps to establish a Karp reduction:

1. Present a computable function $f : \Sigma^* \to \Sigma^*$.
2. Show that $x \in A \Rightarrow f(x) \in B$.
3. Show that $x \notin A \Rightarrow f(x) \notin B$.
4. Show that f can be computed in polynomial time.

- **Theorem**: CLIQUE $\leq_m^P$ IS.

Proof.

```
G = (V, E): graph. k: positive integer
f(<G, k>):
  E' = {{u, v}: {u, v} ∉ E}.
  Output <G' = (V, E'), k>.
```

We argue that $x \in$ CLIQUE iff $f(x) \in$ IS. First, assume $x \in$ CLIQUE. Then x corresponds to a valid encoding $\langle G = (V, E), k\rangle$ of a graph and an integer. Furthermore, G contains a clique $S \subseteq V$ of size k. In the complement graph, this S is an independent set ($\{u, v\} \in E$ for all distinct $u, v \in S \Rightarrow \{u, v\} \notin E'$ for all distinct $u, v, \in S$). Therefore, $\langle G' = (V, E'), k\rangle \in$ IS. Conversely, if $\langle G' = (V, E'), k\rangle \in$ IS, then G' contains an independent set $S \subseteq V$ of size k. This set S is a clique in the complement of G', so the pre-image of $\langle G' = (V, E'), k\rangle$ under f is in CLIQUE.
Finally, we argue that the function f can be computed in poly-time, since the construction of E' can be done in poly-time as there're polynomilally many possible edges. □

- **Theorem**: CIRCUIT-SAT $\leq_m^P$ 3COL.
- *Complexity class* P: We denote by P the set of all languages that can be decided in poly-time, i.e., in time $O(n^k)$ for some constant $k > 0$.
- Let $\mathcal{C}$ be a set of languages containing P.
  - We say that L is $\mathcal{C}$-hard (with respect to Cook reductions) if for all languages $K \in \mathcal{C}, K \leq^P L$. (With respect to poly-time decidability, a $\mathcal{C}$-hard language is at least as "hard" as any language in $\mathcal{C}$.)
  - We say that L is $\mathcal{C}$-complete if L is $\mathcal{C}$-hard and $L \in C$. (A $\mathcal{C}$-complete language represents the "hardest" language in $\mathcal{C}$ with respect to poly-time decidability.)

$\mathcal{C}$-completeness and P: Suppose L is $\mathcal{C}$-complete. Then observe that $L \in \mathsf{P} \Leftrightarrow \mathcal{C} = \mathsf{P}$.

11.2 Exercises

1. Let $G = (V, E)$ be a grpah. A vertex cover in G is a set $C \subseteq V$ such that for every edge $\{x, y\} \in E$, either $x \in C$ or $y \in C$. An independent set in G is a set $S \subseteq V$ such that for any $u, v \in S, \{u, v\} \notin E$. Define the following languages:

$$\text{VERTEX} - \text{COVER} = \{\langle G, k\rangle : G \text{ is a grpah, } k \in \mathbb{N}^+, G \text{ contains a vertex cover of size } k\}$$
$$\text{IND-SET} = \{\langle G, k\rangle : G \text{ is a graph,} k \in \mathbb{N}^+, G \text{ contains an independent set of size } k\}$$

Show that VERTEX-COVER $\leq_m^P$ IND-SET.

- We define a polynomial-time computable function $f : \Sigma^* \to \Sigma^*$

```
def f(x):
  if x is not an encoding <G,k> where G is a graph and k∈ ℕ⁺, return ϵ
  # assuming for our encoding method, ϵ ∉ IND-SET
  let n be the number of vertices in G
  return <G,n-k>
```

Correctness: If $x \in$ VERTEX-COVER then x is a valid encoding $\langle G = (V, E), k\rangle$ where G has n vertices. In addition, there exists a vertex cover of size k. Let S be the subset of vertices in the vertex cover and consider $V \setminus S$. Observe that $V \setminus S$ is an independent set, since for any pair of vertices $u, v \in V \setminus S$, if neither u nor v are in S, then $(u, v) \notin E$. In addition, $|V \setminus S| = n - k$, so there exists an independent set of size $n - k$. Hence $f(x) \in$ IND-SET.

If $f(x) \in$ IND-SET, then G must has an independent set of size $n - k$. Let S be the subset of vertices in the independent set. Similar to previous direction, $V \setminus S$ is a vertex cover. Consider an arbitrary edge $(u, v) \in E$. Because S is an independent set, $u, v \notin S$, or one of $u, v \in V \setminus S$. Thus, all edges are covered by some vertex in $V \setminus S$ so G has a vertex cover of size $n - (n - k) = k$ and $x \in$ VERTEX-COVER.

f is poly-time since we just need to verify the encoding is valid, count the number of vertices and compute $n - k$, which can all be done in poly-time.

Chapter 12

Non-Deterministic Polynomial Time

12.1 Definitions

- *Non-deterministic polynomial time, complexity class* NP: Fix some alphabet Σ. We say that a language L can be decided in *non-deterministic poly-time* if there exists

  1. a poly-time decider TM V that takes two strings as input, and
  2. a constant $k > 0$,

  such that for all $x \in \Sigma^*$:

  - if $x \in L$, then there exists $u \in \Sigma^*$ with $|u| \leq |x|^k$ such that $V(x, u)$ accepts,
  - if $x \notin L$, then for all $u \in \Sigma^*, V(x, u)$ rejects.

  If $x \in L$, a string u that makes $V(x, u)$ accept is called a *proof / certificate* of x being in L. V is called a *verifier*. NP is the set of all languages which can be decided in non-deterministic polynomial time.

- Steps to show a language is in NP:

  1. Present a TM V that takes two inputs x, u.
  2. Argue that V has poly running time.
  3. Argue that V works correctly, which involves arguing the following for some constant $k > 0$:
     - $\forall x \in L, \exists u \in \Sigma^*$ such that $|u| \leq |x|^k, V(x, u)$ accepts;
     - $\forall x \notin L, \forall u \in \Sigma^*, V(x, u)$ rejects.

- **Proposition**: 3COL $\in$ NP

  Proof. To show 3COL is in NP, we need to show that there is a poly-time verifier TM A with the properties stated in the definition. The description of A is as follows.

```
G = (V, E): graph.
A(<G>, u):
  If u is not a valid encoding of a 3-coloring of V, reject.
  If there is {v,w} ∈ E where v and w have the same color, reject.
  Else, accept.

```

  We now show that A satisfies the two conditions stated in the definition. If x is in the language, that means x is a valid encoding of a graph $G = (V, E)$ and this graph is 3-colorable. When u is a valid 3-coloring, $|u| = O(|V|)$. And for this x, u, the verifier accepts. On the other hand, if x is not in the language, then either (i) x is not a valid encoding of a graph or (ii) it is a valid encoding of a graph which is not 3-colorable. In case (i), the verifier rejects (which is doen

implicitly since the input is not of the correct type). In case (ii), any u that does not correspond to a 3-coloring of the vertices makes the verifier reject. Furthermore, any u that does correspond to a 3-coloring of the vertices must be such that there is an edge whose endpoints are colored with the same color. Therefore, in this case, the verifier again rejects as desired.

Now we show that the machine is poly-time. To check whether u is a valid encoding of a 3-coloring of the vertices takes polynomial time since you just need to check that you are given $|V|$ colors, each being one of 3 colors. To check that it is indeed a valid 3-coloring is polynomial time as well since you just need to go through every edge once. □

- CLIQUE, IS, 3SAT $\in$ NP.
- **Proposition**: P $\subseteq$ NP.
- *Complexity class* EXP: We denote by EXP the set of all languages that can be decided in at most exponential-time, i.e., in time $O(2^{n^c})$ for some constant $C > 0$.
  NP $\subseteq$ EXP.
- **Cook-Levin Theorem**: CIRCUIT-SAT is NP-complete.
- **Theorem**: 3COL is NP-complete.
  Theorem: 3SAT is NP-complete.
  Theorem: IS is NP-complete.

12.2 Exercises

1. In MSAT, we want to output True iff the input CNF formula has at least two distinct satisfying assignments. Show that MSAT is NP-complete.

   - MSAT is in NP: we need to show that there is a polynomial time verifier TM V and $t \in \mathbb{N}$ such that
     1. if $x \in$ MSAT, then there is some $u, |u| \leq |x|^t$, such that $V(x, u)$ accepts;
     2. if $x \notin$ MSAT, then for all $u, V(x, u)$ rejects.

```
F: CNF formula
V(<F>, u):
  If u is not a valid encoding of two distinct 0/1 assignments, reject.
  If both assignments in u satisfy F, accept.
  Else, reject.
```

   The verifier is poly-time: checking whether u is a valid encoding of two distinct 0/1 assignments to the input variables can be done in poly time. Given a 0/1 assignment to the input variables, we can plug those values into the input CNF formula, and evaluate the output of the formula. This can be done in poly time. Thus, the verifier has poly-time complexity.

   Correctness: If $x \in$ MSAT, then x must be a valid encoding of a CNF formula F. Furthermore, F must be such that there are at least two distinct 0/1 assignments to the input variables that make F evaluate to 1. When u is the encoding of such two distinct 0/1 assignments, the verifier will accept as desired. Note that clearly for such a $u, |u| \leq O(|F|)$, i.e. the proof is polynomial in the length of F. If $x \notin$ MSAT, then either x is not a valid encoding of a CNF formula (in which case the verifier rejects, which is done implicitly), or x is a valid encoding but there is at most one 0/1 assignment to the input variables that makes x output 1. In either case, if u is not the encoding of two assignments, we reject; and if it is, then the last step detects at least one of assignments makes the formula output 0, so the verifier rejects as desired.

   MSAT is NP-hard: we show a Karp reduction from SAT to MSAT. To show the Karp reduction, we need to show that there is a poly-time computable function $f : \Sigma^* \to \Sigma^*$ such that $x \in$ SAT iff $f(x) \in$ MSAT.

```
F: CNF formula
f(<F>):
  Let z be a variable name not included in F.
  Let F' = F ∧(z ∨ ¬z).
  Output <F'>.
```

Poly time: each step above can be carried out in poly time wrt the length of F.
Correctness: Suppose $x \in$ SAT. Then $x = \langle F \rangle$ for some satisfiable CNF formula F. Let $y_1, \ldots, y_n$ be the variables in F. And suppose $y_1 = b_1, \ldots, y_n = b_n$ is a satisfying assignment for F, where $b_i \in \{0, 1\} \forall i$. Then observe that both $y_1 = b_1, \ldots, y_n = b_n, z = 0$ and $y_1 = b + 1, \ldots, y_n = b_n, z = 1$ are satisfying assignments for F', and o $\langle F' \rangle \in$ MSAT. For the converse, assume $\langle F' \rangle \in$ MSAT. This means $F' = (F \wedge (z \vee \neg z)) = (F \wedge$ True) is satisfiable, which implies F must be satisfiable. So $x = \langle F \rangle \in$ SAT.

2. In the PARTITION problem, we are given n non-negative integers $a_1, a_2, \ldots, a_n$, and we want to output True iff there is a way to partition the integers into two parts so that the sum of the two parts are equal, i.e. there is set $S \subseteq \{1, 2, \ldots, n\}$ such that $\sum_{i \in S} a_i = \sum_{j \in \{1, \ldots, n\} \setminus S} a_j$.
In the BIN problem we are given a set of n items with non-negative sizes $s_1, s_2, \ldots, s_n \in \mathbb{N}$ (not necessarily distinct), a capacity $C \geq 0$ (not necessarily an integer), and an integer $k \in \mathbb{N}$. We want to output True iff the items can be placed into at most k bins such that the total size of items in each bin does not exceed the capacity C.
Show that BIN is NP-complete assuming PARTITION is NP-hard.

- BIN is in NP: we need to show that there is a poly-time verifier TM V and $t \in \mathbb{N}$ such that
  1. if $x \in$ BIN, then there is some $u, |u| \leq |x|^t$ such that $V(x, u)$ accepts;
  2. if $x \notin$ BIN, then for all $u, V(x, u)$ rejects.

```
s_1, ..., s_n, k: non-negative integers. C: non-negative real.
V(<s_1, s_2, ..., s_n, C, k>, u):
  If k > n, set k = n.
  If u is not a valid encoding of a partition (B_1, ..., B_k) of {1, ..., n}, reject.
  If for some j = 1, ..., k, Σ_{i∈B_j} s_i > C, reject.
  Else accept.
```

The verifier is poly time: (First note that the n in the question is wrt the input length, which is $|\langle s_1, s_2, \ldots, s_n, C, k \rangle| + |u|$.) Checking that u is a partition of $\{1, \ldots, n\}$ into k parts, we need to check that there are k sets, their union is $\{1, \ldots, n\}$, and they are pairwise disjoint. All of these can be checked in poly time. In the third step, we implicitly have a loop that repeats $k \leq n$ times. In each iteration, we do at most n additions, which is poly time.
Correctness: If $x \in$ BIN then first, it must be a valid encoding. Furthermore, there must be a partition $(B_1, \ldots, B_k)$ of $\{1, \ldots, n\}$ such that for each $j = 1, \ldots, k, \sum_{i \in B_j} s_i \leq C$. When u represents such a partition, the verifier correctly accepts. Such a u has length poly in n, and therefore poly in the input length as desired.
If $x \notin$ BIN then either x is an invalid encoding (in which case the verifier rejects implicitly) or x is a valid encoding, but there is no way to partition the elements into k bins $(B_1, \ldots, B_k)$ such that $\sum_{i \in B_j} s_i \leq C$ for all bins. If u does not correspond to a partition $(B_1, \ldots, B_k)$ then we reject. If it does, we successfully reject on the third step.
BIN is NP-hard: we show a Karp reduction from PARTITION to BIN. To show the Karp reduction, we need to show that there is a poly-time computable function $f : \Sigma^* \to \Sigma^*$ such that $x \in$ PARTITION iff $f(x) \in$ BIN.

```
a_1, ..., a_n: non-negative integers.
f(<a_1, a_2, ..., a_n>):
  Let s_i = a_i for all i.
  Let T = Σ_{i∈{1,...,n}} s_i.
  Let C = T/2.
  Let k = 2.
  Output <s_1, ..., s_n, C, k>.
```

Poly time: Addition and division are poly-time operations and we only do these operations polynomially many times, so the whole function is poly-time computable.
Correctness: If $x \in$ PARTITION, then $x = \langle a_1, a_2, \ldots, a_n \rangle$ for non-negative integers a_i, and there is $S \subset \{1, \ldots, n\}$ such that $\sum_{i \in S} a_i = \sum_{j \in \{1,\ldots,n\} \setminus S} a_j$. Let $T = \sum_{i \in \{1,\ldots,n\}} a_i$ be the total sum. Then we must have $\sum_{i \in S} a_i = \sum_{j \in \{1,\ldots,n\} \setminus S} a_j = T/2$. If we let $B_1 = S, B_2 = \{1, \ldots, n\} \setminus S$, we see that the elements can be partitioned into two bins of capacity $T/2$. Therefore the output $f(x)$ is indeed an element of BIN.
If $f(x)$ is an element of BIN, then there is a way to partition the elements $s_1 = a_1, \ldots, s_n = a_n$ into $k = 2$ bins B_1 and B_2 ($B_1 \cup B_2 = \{1, \ldots, n\}$, $B_1 \cap B_2 = \varnothing$) of capacity $C = T/2 = \frac{1}{2} \sum_{i \in \{1,\ldots,n\}} s_i$. Since the total sum of the elements is T and each element must be in one of the two bins, the two bins must be completely full, i.e. $\sum_{i \in B_1} a_i = \sum_{j \in B_2} a_j = T/2$, where $S = B_1, \{1, \ldots, n\} \setminus S = B_2$, so $x = \langle a_1, a_2, \ldots, a_n \rangle \in$ PARTITION.

3. By definition, a language $L \subseteq \Sigma^*$ is in NP if for all $x \in \Sigma^*$, there exists a polynomial time TM V and a number k such that if $x \in L$, then there is a u with $|u| \leq |x|^k$ such that $V(x, u) = 1$ and if $x \notin L$ then for all $u, V(x, u) = 0$.

   - False, a language is in NP iff there exists one TM that satisfied above.

4. Every language in NP is decidable.

   - True, since there exists a nondeterministic TM that halts (in poly time) on all inputs with finite length.

5. Let $L \subseteq \Sigma^*$. Define $\mathsf{coNP} = \{L \subseteq \Sigma^* : \overline{L} \in \mathsf{NP}\}$. If $\mathsf{coNP} \neq \mathsf{NP}$, then $\mathsf{P} \neq \mathsf{NP}$.

   - True. We prove by contrapositive: $\mathsf{P} = \mathsf{NP} \Rightarrow \mathsf{coNP} = \mathsf{NP}$.
     $L \in \mathsf{coNP} \Rightarrow \overline{L} \in \mathsf{NP} \Rightarrow \overline{L} \in \mathsf{P} \Rightarrow L \in \mathsf{P}$
     $L \in \mathsf{NP} \Rightarrow L \in \mathsf{P} \Rightarrow \overline{L} \in \mathsf{P} \Rightarrow \overline{L} \in \mathsf{NP} \Rightarrow L \in \mathsf{coNP}$

Chapter 13

Approximation Algorithms

13.1 Definitions

- A *minimization problem* is a function $f : \Sigma^* \times \Sigma^* \to \mathbb{R}^{\geq 0} \cup \{\text{no}\}$. If $f(x, y) = \alpha \in \mathbb{R}^{\geq 0}$,we say that y is a *solution* to x with value α. If $f(x, y) = \text{no}$, then y is not a solution to x. We let $OPT_f(x)$ denote the minimum $f(x, y)$ among all solutions y to x.
  In a *maximization optimization problem*, $OPT_f(x)$ is defined using a maximum rather than a minimum.
  We say that an optimization problem f is computable if there is an algorithm such that given as input $x \in \Sigma^*$, it produces as output a solution y to x such that $f(x, y) = OPT(x)$.

- In the optimization version of VERTEX-COVER **(MIN-VC)**, we are given as input an undirected graph G and the output is a vertex cover of minimum size.
  Let $x = \langle G \rangle$ for some graph G. If y represents a vertex cover in G, then $f(x, y)$ is defined to be the size of the set that y represents; otherwise $f(x, y) = \text{no}$.

- Let f be a minimization optimization problem and let $\alpha > 1$ be some parameter. We say that an algorithm A is an α-approximation algorithm for f if for all instances x, $f(x, A(x)) \leq \alpha \cdot OPT(x)$.
  Let f be a maximization optimization problem and let $0 < \beta < 1$ be some parameter. We say that an algorithm A is a β-approximation algorithm for f if for all instances $x, f(x, A(x)) \geq \beta \cdot OPT(x)$.

- *Lemma*: Given a graph $G = (V, E)$, let $M \subseteq E$ be a matching in G and let $S \subset V$ be a vertex cover in G. Then $|S| \geq |M|$.
  Theorem (Gavril's Algorithm): There is a polynomial-time 2-approximation algorithm for the optimization problem MIN-VC.

  Proof. Our algorithm is as follows:

```
G = (V,E): undirected graph
A(<G>):
  Let M = ∅.
  For each edge e ∈ E do:
    If M ∪ {e} is a matching, let M = M ∪ {e}.
  Let S = set of all the vertices incident to an edge in M.
  Output S.
```

  The running-time is polynomial since we have a loop that repeats $|E|$ times and in each iteration, we do at most $O(|E|)$ steps. So the total cost of the loop is $O(|E|^2)$. The cosntruction of S takes $O(|V|)$ steps, so in total, the algorithm runs in polynomial time.
  Now we argue the algorithm is a 2-approximation algorithm. To do this, we need to argue that

  - S is indeed a valid vertex-cover,
  - if S^* is a vertex cover of minimum size, then $|S| \leq 2|S^*|$.

Notice that M cosntructed by the algorithm is a maximal matching. This implies that the set S is indeed a valid vertex-cover. A convenient lower boudn on $|S^*|$ is given by the lemma: for any matching M, $|S^*| \geq |M|$. Observe that $|S| = 2|M|$. Putting the two together, we get $|S| \leq 2|S^*|$ as desired. □

- Let $G = (V, E)$ be a graph. Given a coloring of the vertices with 2 colors, we say that an edge $e = \{u, v\}$ is a *cut edge* if u, v are colored differently.
  Max-cut problem **(MAX-CUT)**: the input is a graph G and the output is a coloring of the vertices with 2 colors that maximizes the number of cut edges.

- **Theorem**: There is a polynomial-time $\frac{1}{2}$-approximation algorithm for the optimization problem MAX-CUT.

  Proof. The algorithm is as follows:

```
G = (V, E): undirected graph.
A(<G>):
  Color every vertex with the same color. Let c = 0.
  # c stores the number of cut edges
  Repeat:
    If there is a vertex such that changing its color increases the number of cut edges,
    change the color of that vertex. Update c.
    Else, output the current coloring of the vertices.

```

  The algorithm runs in polynomial time. Note that the maximum number of cut edges possible is $|E|$. Therefore the loop repeats at most $|E|$ times. In each iteration, the number of steps we need to take is at most $O(|V|^2)$ since we can just go through every vertex once, and for each one of them, we can check all the edges incident to it. So in total, the number of steps is polynomial in the input length.
  Correctness: it's clear the algorithm returns a valid coloring.
  Since $OPT(\langle G \rangle) \leq m$, we want to show $c \geq \frac{1}{2}m$. Observe that in the coloring that the algorithm returns, for each $v \in V$, at least $\frac{d(v)}{2}$ edges incident to v are cut edges, otherwise, we could change the color of the vertex to obtain a solution that has strictly more cut edges. By Handshake lemma, we have $2c \geq \sum_v \frac{d(v)}{2} = m \Rightarrow c \geq \frac{1}{2}m$ as desired. □

- *Traveling salesperson problem*: the input is a connected graph $G = (V, E)$ together with edge costs $c : E \to \mathbb{N}$. The output is a Hamiltonian cycle that minimizes the total cost of the edges in the cycle, if one exists.
  Metric-TSP: the output is a list of vertices $v_{i_1}, v_{i_2}, \ldots, v_{i_k}, v_{i_1}$ such that the vertices are not necessarily unique, all the vertices of the graph appear in the list, any two consecutive vertices in the list form an edge, and the total cost of the edges is minimized.

- **Theorem**: There is a polynomial-time 2-approximation algorithm for Metric-TSP.

  Proof. The algorithm is as follows:

```
G = (V, E): graph. v : v ∈ V.
A(<G, c>):
  Compute a MST T of G.
  Let v be an arbitrary vertex in V.
  Let L be an empty list.
  Run DFS(<T, v>):
    G = (V, E): graph. v : v ∈ V.
    DFS(<G, v>):
      Mark v as ``visited''.
      Add v to L.
      For each u ∈ N(v):
        If u is not marked ``visited'', then run DFS(<G, u>).
        Add v to L.
  Output L.

```

This is a polynomial-time algorithm since computing a minimum spannign tree and doing a depth-first search both take polynomial time.
Correctness: the MST visits all the vertices and our tour starts and ends at the same vertex v.
Let $c(L)$ denote the total cost of the tour that the algorithm outputs. Let L^* be a optimal solution. Our goal is to show that $c(L) \leq 2c(L^*)$. The graph induced by L^* is connected graph on all of the vertices. Let T^* be a spanning tree within this induced graph, then $c(L^*) \geq c(T^*)$. Since $c(L) = 2c(T)$ as the tour uses every edge of T exactly twice and $c(T) \leq c(T^*)$, then $c(L) \leq 2c(L^*)$ as desired. □

- *Max-coverage problem* **(MAX-COVERAGE)**: the input is a set X, a collection of subsets $S_1, S_2, \ldots, S_m \subseteq X$ and a number $k \in [m]$. The output is a set $T \subseteq [m]$ of size k that maximizes $|\bigcup_{i \in T} S_i|$ (the elements in this union are called *covered elements*).

13.2 Exericises

1. Given the algorithm for the MAX-COVERAGE problem,

```
S_1, S_2, ..., S_m: sets. k: integer in [m].
A(<S_1, ..., S_m, k>):
  T = ∅.
  U = ∅.    # keep track of elements covered
  Repeat k times:
    Pick j such that j ∉ T and |S_j − U| is maximized.
    Add j to T.
    Update U to U ∪ S_j.
  Output T.
```

Let T^* denote the optimum solution and let $U^* = \bigcup_{j \in T^*} S_j$. Note that the value of the optimum solution is $|U^*|$. Define U_i to be the set U in the above algorithm after i iterations of the loop. Let $r_i = |U^*| - |U_i|$. Prove that $r_i \leq \left(1 - \frac{1}{k}\right)^i |U^*|$.

- We prove the claim by induction on the number of iterations.
  - **(BC)** $i = 0$: $r_0 = |U^*| - |U_0| = |U^*| = \left(1 - \frac{1}{k}\right)^0 |U^*|$. (✓)
  - **(IS)** Suppose the algorithm is entering its ith iteration. Let t_{i-1} be the number of uncovered elements in U^* at this point. Note that $t_{i-1} \geq r_{i-1}$ (equality is achieved when all the covered elements are from U^*). We know that there are k sets that do cover all of U^*. This implies at least one of k sets covers at least $\frac{t_{i-1}}{k}$ elements of U^*. At any given iteration, the algorithm picks the set covering the most uncovered elements at the point, so it will pick a set covering at least $\frac{t_{i-1}}{k} \geq \frac{r_{i-1}}{k}$ elements. Therefore,

$$r_i \leq r_{i-1} - \frac{r_{i-1}}{k} = \left(1 - \frac{1}{k}\right) r_{i-1} \leq \left(1 - \frac{1}{k}\right)^i |U^*|$$

Chapter 14

Probability Theory

Chapter 15

Randomized Algorithms

15.1 Definitions

- An algorithm is *randomized* if it has access to a randomness source (using *probabilistic Turing machines*).
  RandInt(m): return a uniformly random element of $\{1, 2, \ldots, m\}$.
  Bernoulli(p): return 1 with probability p and returns 0 with probability $1 - p$.

- *Monte Carlo algorithm*: Let $f : \Sigma^* \to \Sigma^*$ be a computational problem. Let $0 \leq \epsilon < 1$ be some parameter and $T : \mathbb{N} \to \mathbb{N}$ be some function. Suppose A is a randomized algorithm such that

  - $\forall x \in \Sigma^*, \Pr[A(x) \neq f(x)] \leq \epsilon$;
  - $\forall x \in \Sigma^*, \Pr[\text{number of steps } A(x) \text{ takes is at most } T(|x|)] = 1$.

  We say that A is a $T(n)$-time *Monte Carlo algorithm* that computes f with ϵ probability of error.

- *Las Vegas algorithm*: Let $f : \Sigma^* \to \Sigma^*$ be a computational problem. Let $T : \mathbb{N} \to \mathbb{N}$ be some function. Suppose A is a randomized algorithm such that

  - $\forall x \in \Sigma^*, \Pr[A(x) = f(x)] = 1$, where the probability is over the random choices made by A;
  - $\forall x \in \Sigma^*, \mathbb{E}[\text{number of steps } A(x) \text{ takes}] \leq T(|x|)$.

  We say that A is a $T(n)$-time *Las Vegas algorithm* that computes f.

- *Minimum cut problem* **(MIN-CUT)**: the input is a connected undirected graph G, and the output is a 2-coloring of the vertices, where each color is used at least once, such that the number of cut edges is minimized, i.e. we want to output a non-empty subset $S \subseteq V$ such that the number of edges between S and $V \setminus S$ is minimized. Such a set S is called a *cut* and $|S|$ is the number of edges between $S, V \setminus S$.

- A *multi-graph* $G = (V, E)$ is an undirected graph in which E is allowed to be a multi-set, i.e. a multi-graph can have multiple edges between two edges.

- Let $G = (V, E)$ be a multi-graph and $u, v \in V$. *Contraction* of u, v produces a new multi-graph $G' = (V', E')$ in which we remove u, v and create a new vertex uv. $V' = V \setminus \{u, v\} \cup \{uv\}$. The multi-set of edges E' is defined as follows:

  - $\forall \{u, w\} \in E, w \neq v$, we add $\{uv, w\}$ to E';
  - $\forall \{v, w\} \in E, w \neq u$, we add $\{uv, w\}$ to E';
  - $\forall \{w, w'\} \in E, w, w' \notin \{u, v\}$, we add $\{w, w'\}$ to E'.

- **Theorem** (Contraction algorithm for min cut): There is a polynomial-time Monte-Carlo algorithm that solves the MIN-CUT problem with error probability at most $\frac{1}{e^n}$ where n is the number of vertices in the input graph.

  Proof. The algorithm has two phases. The first phase is as follows:

```
G: connected undirected graph
A(<G>):
  Repeat until two vertices remain:
    Select an edge {u,v} uniformly at random.
    Contract u,v to obtain a new graph.
  Two vertices remain, which corresponds to a partition of V into V_1,V_2. Output V_1.
```

Let G_i denote the graph we have after i iterations of the algorithm. So $G_0 = G$. Note that the algorithm has $n-2$ iterations because in each iteration the number of vertices goes down by exactly one and we stop when 2 vertices remain. Then the algorithm runs in polynomial time, since in each iteration contracting an edge can be done in poly time. It leaves to show the success probability of the first phase is at least $\frac{2}{n(n-1)} \geq \frac{1}{n^2}$.
We make two observations:

1. For any i, a cut in G_i of size k corresponds to a cut in $G = G_0$ of size k.
2. For any i and any vertex v in G_i, the size of the minimum cut in G is at most $\deg_{G_i}(v)$. This is because a single vertex v forms a cut by itself and the size of this cut is $\deg(v)$. By observation 1, the original grpah G has a corresponding cut with the same size. Since the minimum cut has the minimum possible size among all cuts in G, its size cannot be larger than $\deg(v)$.

Let $F \subseteq E$ correspond to an optimum solution. We will show

$$\Pr[\text{algorithm finds } F] \geq \frac{2}{n(n-1)}$$

Observe that the algorithm's output corresponds to F iff it never contracts an edge of F. Let E_i be the event that at iteration i, and edge in F is contracted. As noted above, there are $n-2$ iterations in total. Therefore,

$$\begin{aligned}\Pr[\text{algorithm finds } F] &= \Pr[\overline{E}_1 \cap \overline{E}_2 \cap \cdots \cap \overline{E}_{n-2}] \\ &= \Pr[\overline{E}_1] \cdot \Pr[\overline{E}_2 \mid \overline{E}_1] \cdot \Pr[\overline{E}_2 \mid \overline{E}_1 \cap \overline{E}_2] \cdot \cdots \cdot \Pr[\overline{E}_{n-2} \mid \overline{E}_1 \cap \overline{E}_2 \cap \cdots \cap \overline{E}_{n-3}]\end{aligned}$$

First, $\Pr[E_1] = \frac{|F|}{m}$. By observation 2, $\forall v \in V, |F| \leq \deg(v) \Rightarrow 2m = \sum_{v \in V} \deg(v) \geq |F| \cdot n, |F| \leq \frac{2m}{n} \Rightarrow \Pr[E_1] \leq \frac{2}{n} \Rightarrow \Pr[\overline{E}_1 \geq 1 - \frac{2}{n}]$.
Then $\Pr[\overline{E}_2 \mid \overline{E}_1] = 1 - \Pr[E_2 \mid \overline{E}_1] = 1 - \frac{|F|}{\ell_1}$ where ℓ_1 is the number of edges remaining after the first iteration. Similarly, $2\ell_1 \geq |F|(n-1) \Rightarrow \Pr[\overline{E}_2 \mid \overline{E}_1] \geq 1 - \frac{2|F|}{|F|(n-1)} = 1 - \frac{2}{n-1}$.
Applying the same reasoning for the rest of the terms in the product above, we get

$$\begin{aligned}\Pr[\text{algorithm finds } F] &\geq \left(1 - \frac{2}{n}\right) \cdot \left(1 - \frac{2}{n-1}\right) \cdot \left(1 - \frac{2}{n-2}\right) \cdots \left(1 - \frac{2}{n-(n-3)}\right) \\ &= \left(\frac{n-2}{n}\right) \cdot \left(\frac{n-3}{n-1}\right) \cdot \left(\frac{n-4}{n-2}\right) \cdot \left(\frac{2}{4}\right) \cdot \left(\frac{1}{3}\right) = \frac{2}{n(n-1)} = \frac{1}{\binom{n}{2}} \geq \frac{1}{n^2}\end{aligned}$$

In the second phase, we boost the success probability by repeating the first phase t times using completely new and independent random choices. Among the t cuts we find, we return the minimum-sized one. As t grows, the success probability increases. Let $t = n^3$. Let A_i be the event that our algorithm doesn't find a min-cut at repetition i. Note that the A_i's are independent since our algorithm uses fresh random bits for each repetition. Also each A_i has the same probability, i.e. $\Pr[A_i] = \Pr[A_j], \forall i, j$. Therefore,

$$\Pr[\text{our algorithm fails to find a min-cut}] = \Pr[A_1 \cap \cdots \cap A_t] = \Pr[A_1] \ldots \Pr[A_t] = \Pr[A_1]^t$$

From the analysis of the first phase, we know that $\Pr[A_1] \leq 1 - \frac{1}{n^2}$, so $\Pr[\text{our algorithm fails to find a min-cut}] \leq \left(1 - \frac{1}{n^2}\right)^t$. To upper bound this, we use the fact that

$$\forall x \in \mathbb{R}, 1 + x \leq e^x$$

Then $\Pr[\text{our algorithm fails to find a min-cut}] \leq (1+x)^t \leq e^{xt} = e^{-t/n^2} = \frac{1}{e^n}$ as desired. $\square$

15.2 Exercises

1. Suppose you're given a Las Vegas algorithm A that solve $f : \Sigma^* \to \Sigma^*$ in expected time $T(n)$. Show that for any constant $\epsilon > 0$, there is a Monte Carlo algorithm that solves f in time $O(T(n))$ and error probability ϵ.

   - Given the Las Vegas algorithm A and a constant $\epsilon > 0$, we construct a Monte Carlo algorithm A' with the desired properties as follows:

```
x: string.
A'(x):
  Run A(x) for 1/ε T(|x|) steps.
  If A terminates, return its output.
  Else, return ``failure''.

```

   Since ϵ is a constant, the running time of A' is $O(T(n))$. Then error probability of the algorithm can be bounded using Markov's inequality as follows. For any $x \in \Sigma^*$, let $\mathbf{T}_x$ be the random variable that denotes the number of steps $A(x)$ takes. Note that by the definition of Las Vegas algorithm, $\mathbb{E}[\mathbf{T}_x] \leq T(|x|)$ for all x. In the event of $A'(x)$ failing, it must be the case that $\mathbf{T}_x > \frac{1}{\epsilon}T(|x|)$. So the probability that $A'(x)$ fails can be upper bounded by

$$\Pr\left[\mathbf{T}_x > \frac{1}{\epsilon}T(|x|)\right] \leq \Pr\left[\mathbf{T}_x \geq \frac{1}{\epsilon}\mathbb{E}[\mathbf{T}_x]\right] \leq \epsilon$$

2. (Boosting for two-sided error) Let $f : \Sigma^* \to \{0, 1\}$ be a decision problem, and let A be a Monte Carlo algorithm for f with error probability $1/4$, i.e. $\forall x \in \Sigma^*, \Pr[A(x) \neq f(x)] \leq 1/4$. We want to boost the success probability to $1 - 1/2^n$, and out strategy will be as follows. Given x, run $A(x)$ $6n$ times (where $n = |x|$), and output the more common output bit among the $6n$ output bits (breaking ties arbitrarily). Show that the probability of outputting the wrong answer is at most $1/2^n$.

   - Let X_i be a Bernoulli indicator RV corresponding to whether the algorithm gives the correct answer in iteration i. Let $X = \sum_{i=1}^{6n} X_i$. So $X \sim \text{Bin}(6n, 3/4)$. Note that

$$\begin{aligned}
\Pr[X = i] &= \binom{6n}{i}\left(\frac{3}{4}\right)^i\left(\frac{1}{4}\right)^{6n-i} \\
\Pr[\text{error}] \leq \Pr[X \leq 3n] \leq \sum_{i=0}^{3n}\Pr[X = i] &= \sum_{i=0}^{3n}\binom{6n}{i}\left(\frac{3}{4}\right)^i\left(\frac{1}{4}\right)^{6n-i} \\
&= \sum_{i=0}^{3n}\binom{6n}{i}\frac{3^i}{4^{6n}} \\
&\leq \sum_{i=0}^{3n}\binom{6n}{i}\frac{3^{3n}}{4^{6n}} \\
&= \frac{3^{3n}}{4^{6n}} \cdot \sum_{i=0}^{3n}\binom{6n}{i} \\
&\leq \frac{3^{3n}}{4^{6n}} \cdot 2^{6n} = \frac{27^n}{64^n} < \frac{1}{2^n}
\end{aligned}$$

3. Using the analysis of the randomized minimum cut algorithm, show that a graph can have at most $n(n-1)/2$ distinct minimum cuts.

   - Suppose there are t distinct minimum cuts $F_1, F_2, \ldots, F_t$. Our goal is to show $t \leq \binom{n}{2}$. Let A_i be the even that the first phase of our algorithm outputs F_i. We know that for any $i, \Pr[A_i] \leq \frac{1}{\binom{n}{2}}$. Furthermore, the events A_i are disjoint, so $\Pr[A_1 \cap A_2 \cap \cdots \cap A_t] = \Pr[A_1] + \Pr[A_2] + \cdots + \Pr[A_t] \geq \frac{t}{\binom{n}{2}}$. Since $\Pr[A_1 \cap A_2 \cap \cdots \cap A_t] \leq 1$, we can conclude $t \leq \binom{n}{2}$.

Chapter 16

Markov Chains

16.1 Definitions

- A *Markov chain* is a directed graph with n vertices with the following properties:
  - each directed edge is labed with a non-zero probability $p \in (0, 1]$;
  - for each vertex i, the sum of the probabilities on edges leaving i is 1.

  The nodes of the graph are called *states*. The edges are called *transitions*.
  A Markove chain can be equivalently described using an $n \times n$ *transition matrix* K in which each entry is a probability between 0 and 1, and the sum of the values in each row is 1. The entry $K[i, j]$ corresponds to the probability for the edge (i, j).

- Given a Markove chain, $\pi_t = \pi_0 K^t$.
  $\Pr[i \to j \text{ in exactly } t \text{ steps}] = K^t[i, j]$.

- A Markov chain is called *irreducible* if for every two vertices i, j, there is a path from i to j.

- Given a Markov chain, we say that state i is *aperiodic* if there exists $t > 0$ such that for all $t' \geq t$, $\Pr[\mathbf{X}_{t'} = i \mid \mathbf{X}_0 = i] > 0$.
  A Markove chain is called aperiodic if all of its states are aperiodic.

- **Theorem (Fundamental Theorem of Markov Chains)**: Consider a Markov chain that is irreducible and aperiodic.
  - There exists a unique probability distribution π such that $\pi K = \pi$. This distribution is called the *invariant / stationary distribution*.
  - For any initial distribution π_0 over the set of states we have $\forall i, \lim_{t\to\infty} \pi_0 K^t[i] = \pi[i]$ where π is the stationary distribution. In other word, π_t converges to π as $t \to \infty$.
  - Let $\mathbf{T}_{ij}$ be the random variable corresponding to the number of steps it takes to reach j (for the first time) given that we start at i. Then $\mathbb{E}[\mathbf{T}_{ij}] = \frac{1}{\pi[i]}$. This quantity is known as the *mean recurrence time*.

- $\pi = \left[\frac{d(1)}{2m}, \frac{d(2)}{2m}, \ldots, \frac{d(n)}{2m}\right]$ is the stationary distribution for the Markov chain, then $\mathbb{E}[\mathbf{T}_{ii}] = \frac{2m}{d(i)}$.

- *Lemma*: Let $G = (V, E)$ be a connected undirected graph. Let $\{o.j\} \in E$ be an edge in G. Then $\mathbb{E}[\mathbf{T}_{ij}] \leq 2m$.

  Proof. Let $j_1, j_2, \ldots, j_k$ be the neighbors of vertex j. AWLOG that $i = j_1$. By the law of total expectation, we have

  $$\frac{2m}{d(j)} = \mathbb{E}[\mathbf{T}_{jj}] = \sum_{r=1}^{k} \Pr[j \to j_r \text{ in one step}]\mathbb{E}[\mathbf{T}_{jj} \mid \text{ first step is } j \to j_r] = \sum_{r=1}^{k} \frac{1}{d(j)}(1 + \mathbb{E}[\mathbf{T}_{j_r j}]) \geq \frac{1}{d(j)}(1 + \mathbb{E}[\mathbf{T}_{ij}])$$

  so $2m \geq 1 + \mathbb{E}[\mathbf{T}_{ij}] > \mathbb{E}[\mathbf{T}_{ij}]$. □

- **Theorem**: Let $G = (V, E)$ be a connected undirected graph. Then for any $i, j, \mathbb{E}[\mathbf{T}_{ij}] \leq n^3$.

- In the *undirected connectivity problem*, we are given as input an undirected graph $G = (V, E)$ and two vertices $s, t \in V$. The output is "yes" if there is a path from s to t in G and "no" otherwise.
- **Theorem**: There is a polynomial-time randomized Monte Carolo algorithm that solves the undirected connectivity problem using $O(\log n)$ space and ϵ probability error for some constant $\epsilon > 0$.

16.2 Exercises

1.

Chapter 17

Modular Arithmetic

17.1 Definitions

- We let $\mathbb{Z}_N$ denote the set $\{0, 1, 2, \ldots, N-1\}$.
- In $\mathbb{Z}_N$, the element 0 is called the *additive identity* such that for any $A \in \mathbb{Z}_N, A +_N 0 = 0 +_N A = A$.
  The *additive inverse* of A, denoted as $-A$, is defined to be an element in $\mathbb{Z}_N$ such that $A +_N -A = 0$.
  Let $A, B \in \mathbb{Z}_N$. We define "A minus B", denoted $A -_N B$ as $A +_N -B$.
  In the addition table of $\mathbb{Z}_N$, every row and column is a permutation of the elements $\mathbb{Z}_N$.
- Let $A \in \mathbb{Z}_N$. The *multiplicative inverse* of A, denoted as A^{-1} is defined to be an element in $\mathbb{Z}_N$ such that $A \cdot_N A^{-1} = 1$.
  Let $A, N \in \mathbb{N}$. The multiplicative inverse of A in $\mathbb{Z}_N$ exists iff $\gcd(A, N) = 1$.
  Let $A, B \in \mathbb{Z}_N$, where B has a multiplicative inverse B^{-1}.
  Then we define "A divided by B", denoted $A/_N B$, as $A \cdot_N B^{-1}$.
- We let $\mathbb{Z}_N^*$ denote the set $\{A \in \mathbb{Z}_N : \gcd(A, N) = 1\}$. In other words, $\mathbb{Z}_N^*$ is the set of all elements of $\mathbb{Z}_N$ that have a multiplicative inverse.
  $\mathbb{Z}_N^*$ is closed under multiplication.
- The Euler totient function $\varphi : \mathbb{N} \to \mathbb{N}$ is defined as $\varphi(N) = |\mathbb{Z}_N^*|$. By convention, $\varphi(0) = 0$.
- **Euler's Theorem**: For any $A \in \mathbb{Z}_N^*, A^{\varphi(N)} = 1$.

  Proof. Take an arbitrary $A \in \mathbb{Z}_N^*$. Let $B_1, \ldots, B_k$ be the elements of $\mathbb{Z}_N^*$ where $k = \varphi(N)$. By Multiplication table permutation property, $\{AB_1, \ldots, AB_k\} = \mathbb{Z}_N^*$. Then $(AB_1) \ldots (AB_k) = B_1 \ldots B_k$. Multiplying both sides by the inverse of $B_1 \ldots B_k$, we get $A^k = 1$ as desired. □
- If we're exponentiating an element $A \in \mathbb{Z}_N^*$, then we can effectively think of the exponenet as living in the set $\mathbb{Z}_{\varphi(N)}$.
- Let $A \in \mathbb{Z}_N^*$. We say that A is a *generator* if $\{A^E : E \in \mathbb{Z}_{\varphi(N)}\} = \mathbb{Z}_N^*$.
- **Theorem**: If P is a prime number, then $\mathbb{Z}_P^*$ contains a generator.
- Let $A, B, C \in \mathbb{N}$. We say that C is a *miix* of A, B if $C = kA + \ell B$ for some $k, \ell \in \mathbb{Z}$.
- *Fast modular exponentiation* $\mathrm{FME}(A, E, N)$:
  - Repeatedly square A to obtain $A^2 \mod N, \ldots, A^{2^n} \mod N$.
  - Multiply together the powers of A (modulo N) so that the product is A^E. To figure out which powers to multiply, look at the binary representation of E.
  - Taking roots: Given $A, E, N \in \mathbb{N}$ with $A \in \mathbb{Z}_N^*$, the goal is to output $B \in \mathbb{N}$ such that $B^E \equiv_N A$: unknown poly-time computable $\Rightarrow$ RSA cryptosystem.
    Taking logarithms: Given $A, B, P \in \mathbb{N}$, P is a prime, $A \in \mathbb{Z}_P^*$ and $B \in \mathbb{Z}_P^*$ is a generator, the goal is to output $X \in \mathbb{N}$ such that $B^X \equiv_P A$: unknown poly-time computable $\Rightarrow$ Diffie-Hellman secret key exchange protocol.

17.2 Exercises

1. (Multiplication table permutation property) Show that in the multiplication table of $\mathbb{Z}_N^*$, every row and column is a permutation of the elements $\mathbb{Z}_N^*$.

   - AFSOC two entries of the same row are the same. Then there exists $B, B' \in \mathbb{Z}_N^*, B \neq B'$ such that $A \cdot_N B = A \cdot_N B'$. But then if we multiply both sides of the equality by A^{-1} we get $B = B'$, contradiction.

2. Show that for P a prime number, $\varphi(P) = P-1$. Also show that for P, Q distinct prime numbers, $\varphi(PQ) = (P-1)(Q-1)$.

   - If P is a prime number, then for any $A \in [P-1]$, we have $\gcd(A, P) = 1 \Rightarrow \varphi(P) = P - 1$. When $N = PQ$ for distinct primes P, Q, the elements that are not relatively prime to PQ are 0 and $mP, m \in [Q-1], nQ, n \in [P-1]$. In total there are $1 + (Q-1) + (P-1) = Q + P - 1$ of these elements. Then the number of elements that are relatively prime to PQ is $PQ - (Q + P - 1) = PQ - Q - P + 1 = (P-1)(Q-1)$.

Chapter 18

Group Theory

Chapter 19

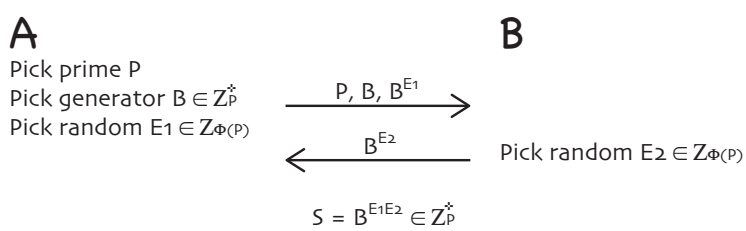
Cryptography

19.1 Definitions

- In a private-key cryptographic system, a protocal is designed as follows: assume A has a private key K_A and B has a private key K_B. If M is the *plaintext* that A wants to send, an encryption algorithm Enc is used to produce a *ciphertext* $C = Enc(M, K_A)$, which is sent to B. Then B uses a decryption algorithm Dec and produces a plaintext $M = Dec(C, K_B)$.

- *One-time pad*: assume the plaintext $M \in \{0,1\}^n$ is a binary string of length n, choose $K := K_A = K_B \in \{0,1\}^n$ uniformly at random. The encryption algorithm takes the bit-wise xor: $C = M \oplus K$. The decryption algorithm is the same as the encryption: $M = C \oplus K = (M \oplus K) \oplus K = M \oplus (K \oplus K)$.
  A and B have to share a key of the same length of the message.
  Key cannot be reused.
  If K is shorter than M and the attacker is computationally unbounded, then it can learn some information about M.

- *Diffie-Hellman secret key exchange*: assume the Discrete Log problem is computationally hard, A picks privately a (sufficiently large) random prime P, a generator $B \in \mathbb{Z}_P^*$, a random exponenet $E_1 \in \mathbb{Z}_{\varphi(P)}$. A sends (P, B, B^{E_1}) to B, and B picks random $E_2 \in \mathbb{Z}_{\varphi(P)}$ and sends B^{E_2} to A. Then both can privately compute the secret key $S = B^{E_1 E_2}$.
  Diffie-Hellman assumption: given P, B, B^{E_1}, B^{E_2}, it is computationally hard to compute $B^{E_1 E_2}$.



- A function $\text{negl}(n)$ is called negligible if for every polynomial $P(n)$ and for all large enough n, $\text{negl}(n) < \frac{1}{P(n)}$.

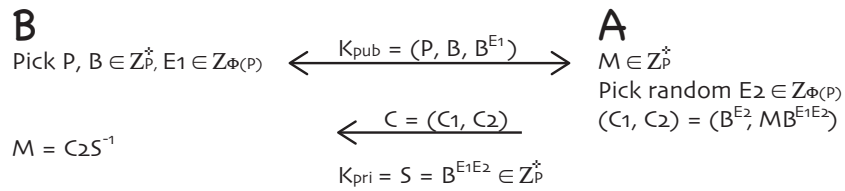
- *Discrete Log Assumption (DLA)*: Let G be a cyclic group with q elements, $g \in G$ be a generator. Then for every probabilistic poly-time algorithm A, $\Pr_{x \in_R \mathbb{Z}_q}[A(g^x) = x] \leq \text{negl}(|q|)$.

- *Computational Diffie-Hellman Assumption (CDH)*: Let G be a cyclic group with q elements, $g \in G$ be a generator. Then for every probabilistic poly-time algorithm A, $\Pr_{x,y \in_R \mathbb{Z}_q}[A(g^x, g^y) = g^{xy}] \leq \text{negl}(|q|)$.
  CDH is weaker than DLA.

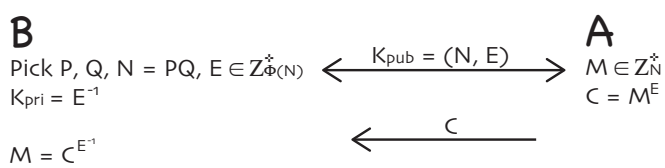
- *Decisional Diffie-Hellman Assumption (DDH)*: Let G be a cyclic group with q elements, $g \in G$ be a generator. Then for every probabilistic poly-time algorithm A, there is no poly-time algorithm that can distinguish between (g^{xy}, g^x, g^y) and (g^r, g^x, g^y) where $r, x, y, \in_R \mathbb{Z}_q$.
  DDH is weaker than CDH.

- *Public-key cryptographic system*: B generates a tuple of keys $(K_{\text{pri}}, K_{\text{pub}})$ where K_{pri} is called the *private keys* and K_{pub} is called the *public key* are is published to the world. If A wants to send a message to B, it runs the encryption algorithm to get $C = \text{Enc}(M, K_{\text{pub}})$ and B can decrypt it by running a decryption algorithm to get $M = \text{Dec}(C, K_{\text{pri}})$.

- *ElGamal public-key cryptographic system*: B picks a (sufficiently large) prime number P, a generator $B \in \mathbb{Z}_P^*$ and a random exponent $E_1 \in \mathbb{Z}_{\varphi(P)}$. The private key is $K_{\text{pri}} = E_1$ and the public key is $K_{\text{pub}} = (P, B, B^{E_1})$. The message $M \in \mathbb{Z}_P^*$. A picks a random $E_1 \in \mathbb{Z}_{\varphi(P)}$ and sends the ciphertext $C = (C_1, C_2) = (B^{E_2}, MB^{E_1E_2})$. B first computes $S = C_1^{E_1} = B^{E_1E_2} \in \mathbb{Z}_P^*$, which is the *secret key* from the Diffie-Hellman protocol. Then it computes $S^{-1} \in \mathbb{Z}_P^*$ and $M = C_2S^{-1} = M$.



- *RSA public-key crptographic system*: assuming that taking roots in teh modular universe is computationally hard, B picks two (sufficiently large) distinct prime P, Q to get $N = PQ$ and an exponent $E \in \mathbb{Z}_{\varphi(N)}^*$. Then $K_{\text{pri}} = E^{-1} \in \mathbb{Z}_{\varphi(N)}^*, K_{\text{pub}} = (N, E)$. The message $M \in \mathbb{Z}_N^*$. A encrypts by $C = M^E \in \mathbb{Z}_N^*$. B decrypts by $M = C^{E^{-1}} = (M^E)^{E^{-1}} = M^{EE^{-1}} \in \mathbb{Z}_N^*$.



- *Commitment schemes* are protocols that allow a committer to send a "locked" message which the receiver cannot "unlock". In the future, the same committer can provide a key to "unlock" the message, allowing the receiver to read while guaranteeing that the committer couldn't have chosen a new key ad-hoc.

  1. *Commit phase*: Given a message m, the committer sends to the receiver the value $c = Com(m, s)$ for some random key s and some poly-time algorithm Com.
  2. *Decommit phase*: The committer then sends the pair (m, s); the receiver verifies that the pair is indeed correct by computing $Com(m, s)$ and comparing it with the previously received c. If they're equal, the receiver accepts, and otherwise, the receiver rejects.

  A commitment scheme must satisfy the following properties:

  1. *Hiding*: At the end of the commit phase, the receiver doesn't learn anything about the message m.
  2. *Binding*: Once the commit phase is finished, the sender is only left with 1 possible message m which he can use in the decommit phase.

- *El-Gamal commitment scheme*: assuming DDH, given a group G with order q and generator $g \in G$:

  1. Commit phase: Given $m \in G$, the committer generates $s = (a, b)$ with $a, b \in \mathbb{Z}_q$ and sends $c = (g^a, g^b, mg^{ab})$
  2. Decommit phase: The committer sends $m, (a, b)$, and the receiver checks if c matches (g^a, g^b, mg^{ab})

  The binding property requires that each message uniquely defines a c. Since a, b are fixed, g^{ab} and thus mg^{ab} is fixed.

- *Coin Flipping by Telephone (or Internet)* is a protocol for establishing consensus on randomness between two parties without requiring them to trust the other party or verify their means of generating the randomness. Consider two parties A and B running the following protocol:

1. A generates r_1 uniformly and sends $Com(r_1, s)$ to B.
2. B generates r_1' uniformly and sends r_1' to A.
3. A decommits by sending r_1, s to B.
4. A, B both output $r = r_1 \oplus r_1'$ as their agreed uniform randomness.

At the end of this coin flipping protocol, $r = r_1 \oplus r_1'$ is uniformly random as long as at least one of the parties is honest.

- A function $f : \{0,1\}^n \to \{0,1\}^m$ is a *one-way function* if

  1. We can compute $f(x)$ in polynomial time.
  2. For any probabilistic polynomial-time adversary A, there exists a negligible function $\text{negl}(n)$ such that

$$\Pr_{x \in_R \{0,1\}^n, x' \in_R A(f(x))}[f(x) = f(x')] \leq \text{negl}(n)$$

- *Digital Signature Syntax*: Let n be a parameter called the *security parameter*. A Digital Signature scheme consists of three probabilistic polynomial-time algorithms:

  – $\text{Gen}(n)$ outputs a secret key and a public key (s_k, p_k).
  – $\text{Sign}(s_k, m)$ outputs σ as the signature of m given a message m and the secret key s_k.
  – $\text{Verify}(p_k, m, \sigma)$ outputs 0/1 given a message m, the signature of m, and the public key p_k.

  Correctness: let m be an arbitrary message, (s_k, p_k) be the output of $\text{Gen}(n)$ and let σ be the output of $\text{Sign}(s_k, m)$. Then $\Pr[\text{Verify}(p_k, \sigma, m) = 1] = 1$.

- *Security of Digital Signature (Unforgeability)*: A challenger C and an adversary A play the following forging game:

  – (Keygen): C samples a key pair $\text{Gen}(n) \to (s_k, p_k)$ and sends p_k to A.
  – (Learning): for $1 \leq i \leq l = poly(n)$, A sends a message m_i to C and receives $\sigma_i = sign(s_k, m_i)$.
  – (Guessing): A outputs (m, σ) and wins if $m \notin \{m_1, \ldots, m_l\}$ and $\text{Verify}(p_k, m, \sigma) = 1$

  For all probabilistic polynomial-time (PPT) algorithm A, the chance of winning the forging game is negligible:

$$(\forall \text{PPT A}) \quad \Pr[A \text{ wins}] \leq \text{negl}(n)$$

- *Security of One-time Signatures*: same as unforgeability except that A sends a single message m to C.